Code Generation

1

Code Generation at a Glance

- At this point, we have optimized IR code that needs to be converted into the target language (e.g. assembly, machine code).
- Goal of this stage:
  - Choose the appropriate machine instructions for each IR instruction.
  - Allocate finite machine resources (registers, caches, etc.)
  - Implement low-level details of the runtime environment.
  -
- Machine-specific optimizations are often done here, though some are treated as part of a final optimization phase.

2

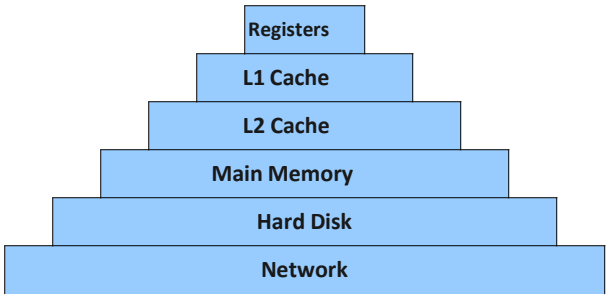
Overview

- **Register Allocation**
  - How to assign variables to finitely many registers? What to do when it can't be done? How
  - to do so efficienty?
  -
- **Garbage Collection**
  - How to detect reclaimable memory? How to
  - reclaim memory efficiently?

3

The Memory Hierarchy

- **Idea**: Try to get the best of all worlds by using multiple types of memory.



256B - 8KB	0.25 – 1ns
16KB – 64KB	1ns – 5ns
1MB - 4MB	5ns – 25ns
4GB – 256GB	25ns – 100ns
500GB+	3 – 10ms
HUGE	10 – 2000ms

4

Registers

- Most machines have a set of **registers**, dedicated memory locations that
  - can be accessed quickly,
  - can have computations performed on them, and exist in
  - small quantity.
- Using registers intelligently is a critical step in any compiler.
  - A good register allocator can generate code orders of magnitude better than a bad register allocator.

5

Register Allocation

- There are an unlimited number of variables.
- On a physical machine there are a small number of registers:
  - x86 has four general-purpose registers and a number of specialized registers.
  - MIPS has twenty-four general-purpose registers and eight special-purpose registers.
- **Register allocation** is the process of assigning variables to registers and managing data transfer in and out of registers.

6

Challenges in Register Allocation

- **Registers are scarce**.
  - Often substantially more IR variables than registers. Need to find a way
  - to reuse registers whenever possible.
- **Registers are complicated**.
  - x86: Each register made of several smaller registers; can't use a register and its constituent registers at the same time.
  - x86: Certain instructions must store their results in specific registers; can't store values there if you want to use those instructions.
  - MIPS: Some registers reserved for the assembler or operating system.
  - Most architectures: Some registers must be preserved across function calls.
  -

7

An Initial Register Allocator

- **Idea**: Store every value in main memory, loading values only when they're needed.
- To generate a code that performs a computation:
  - Generate **load** instructions to pull the values from main memory into registers.
  - Generate code to perform the computation on the registers.
  - Generate **store** instructions to store the result back into main memory.

8

Building a Better Allocator

- **Goal**: Try to hold as many variables in registers as possible.
  - Reduces memory reads/writes.
  - Reduces total memory usage.
- We will need to address these questions:
  - Which registers do we put variables in? What do we do
  - when we run out of registers?

9

Register Consistency

- At each program point, each variable must be in the same location.
  - Does **not** mean that each variable is always stored in the same location!
- At each program point, each register holds at most one live variable.
  - Can assign several variables the same register if no two of them ever will be read together.

10

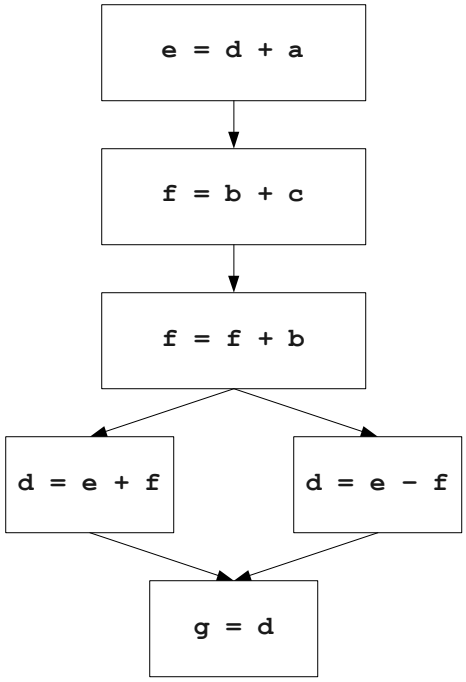
Live Ranges and Live Intervals

- Recall: A variable is **live** at a particular program point if its value may be read later before it is written.
  - Can find this using global liveness analysis.
- The **live range** for a variable is the set of program points at which that variable is live.
- The **live interval** for a variable is the smallest subrange of the IR code containing all a variable's live ranges.
  - A property of the IR code, **not** the CFG.
  - Less precise than live ranges, but simpler to work with.

11

Live Ranges and Live Intervals

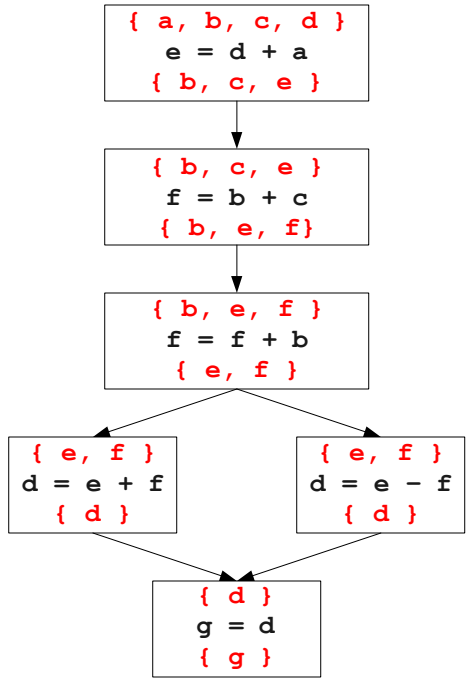
```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



12

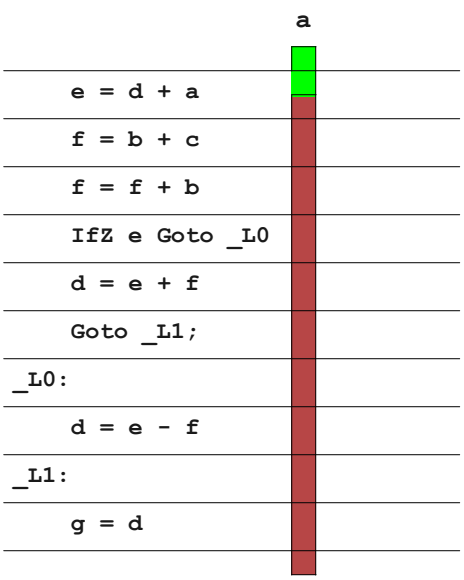
Live Ranges and Live Intervals

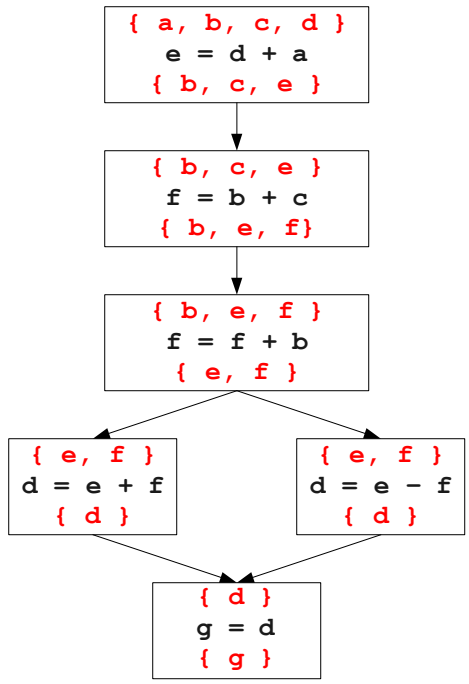
```
    e = d + a
    f = b + c
    f = f + b
    IfZ e Goto _L0
    d = e + f
    Goto _L1;
_L0:
    d = e - f
_L1:
    g = d
```



13

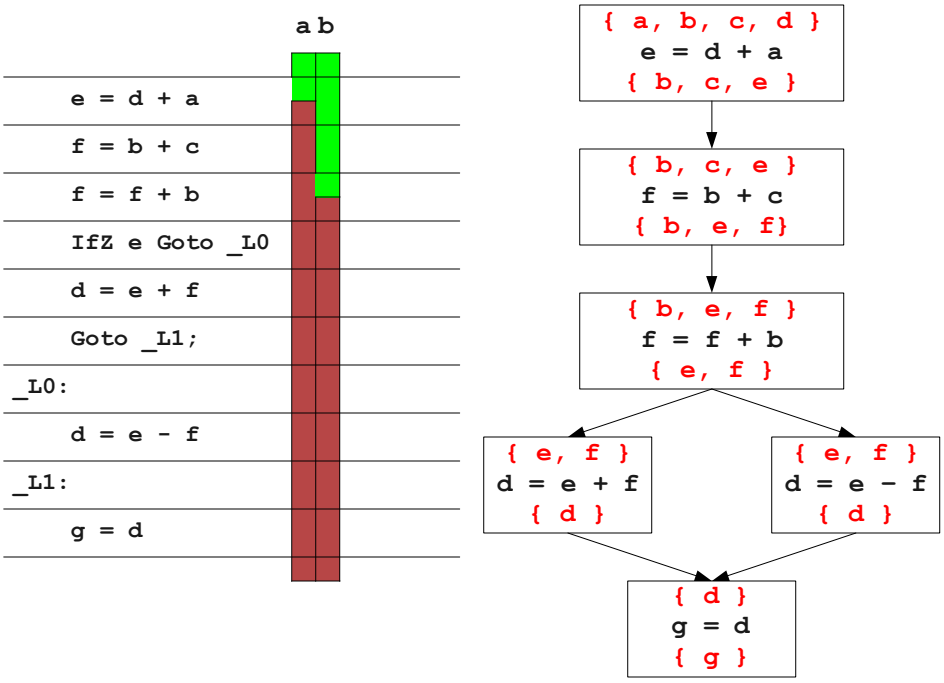
Live Ranges and Live Intervals





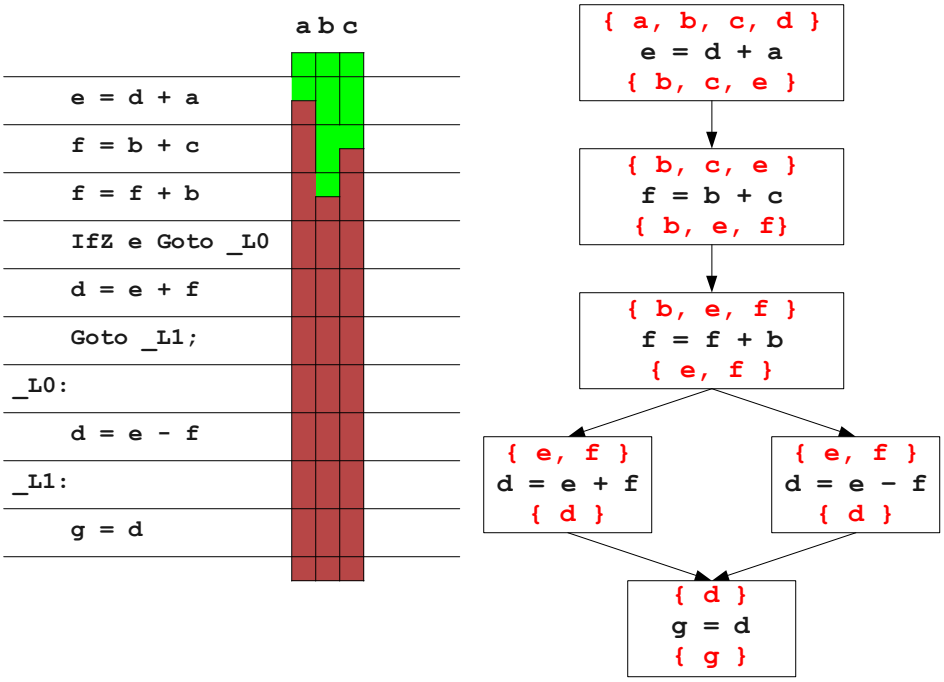
14

Live Ranges and Live Intervals



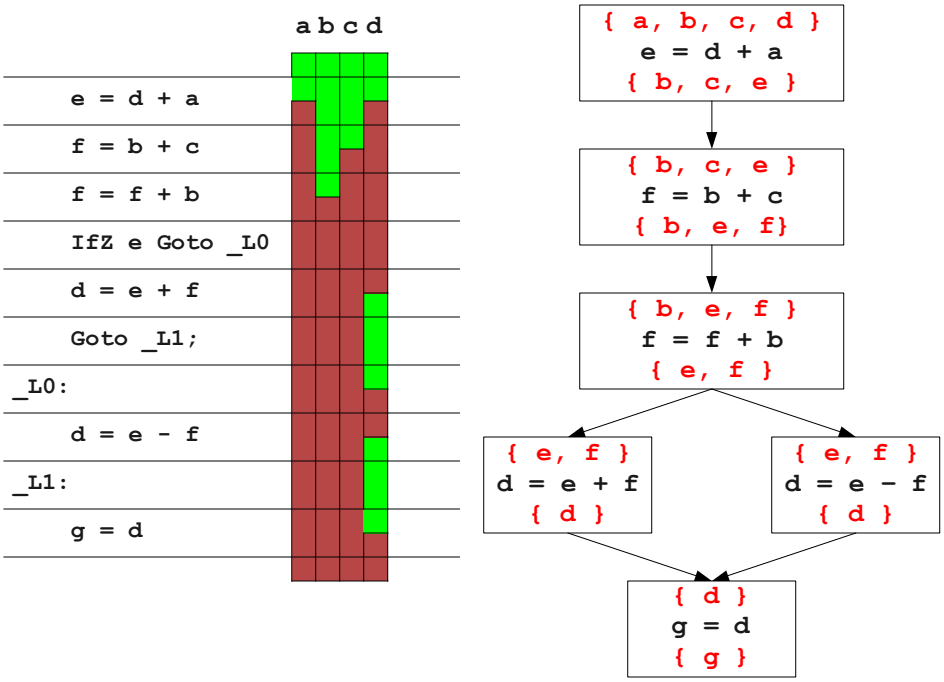
15

Live Ranges and Live Intervals



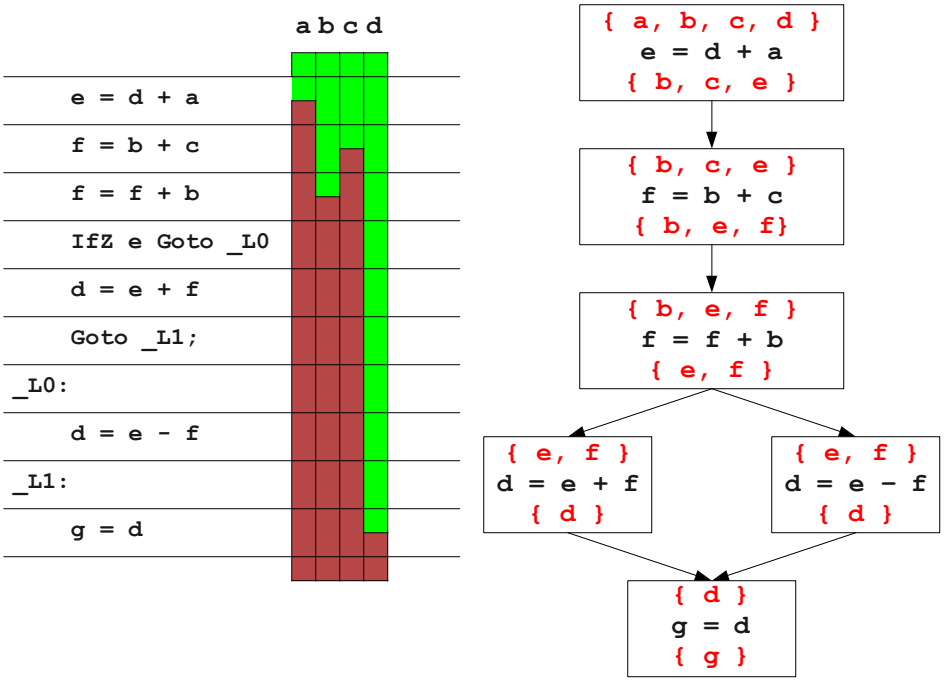
16

Live Ranges and Live Intervals



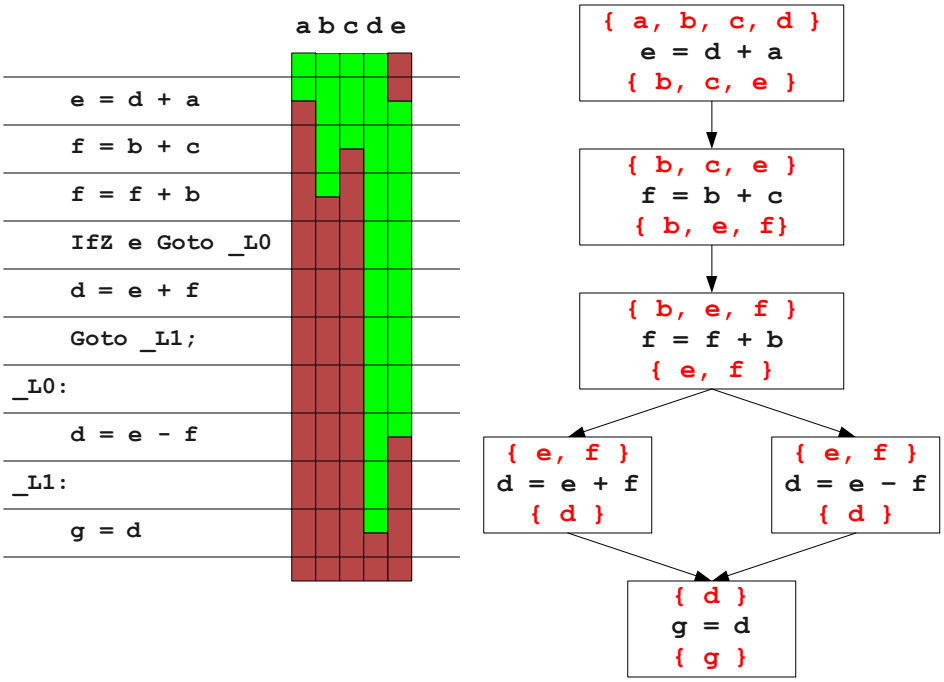
17

Live Ranges and Live Intervals



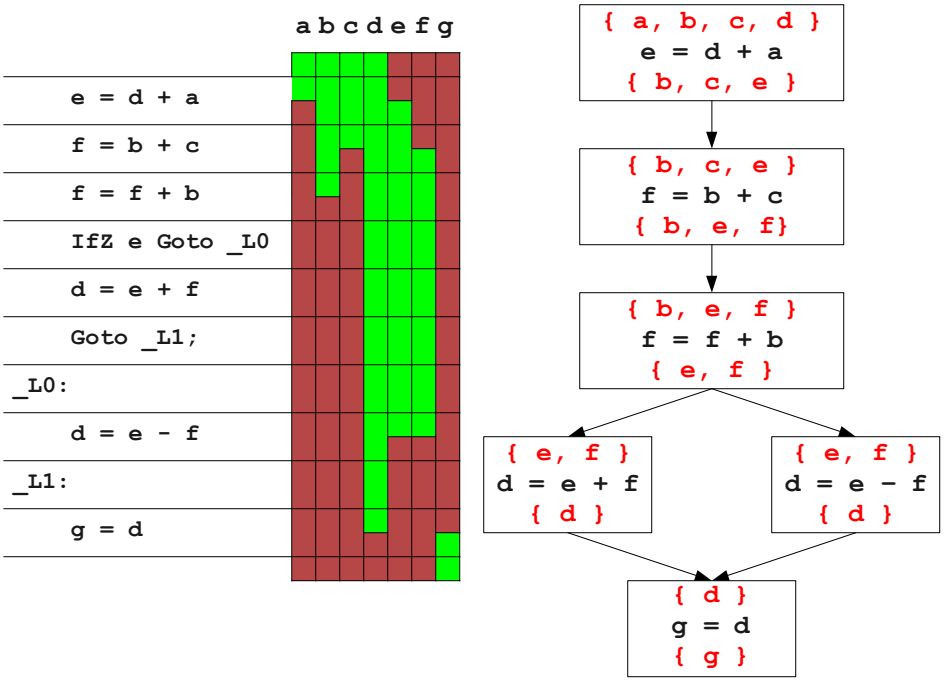
18

Live Ranges and Live Intervals



19

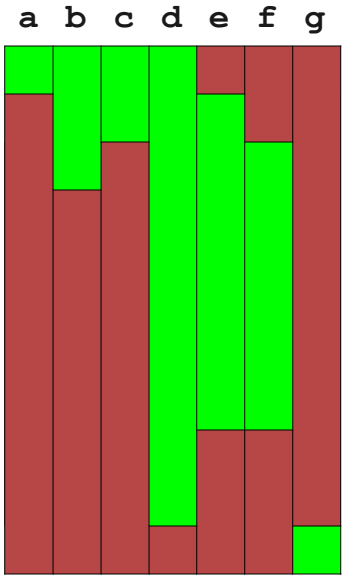
Live Ranges and Live Intervals



20

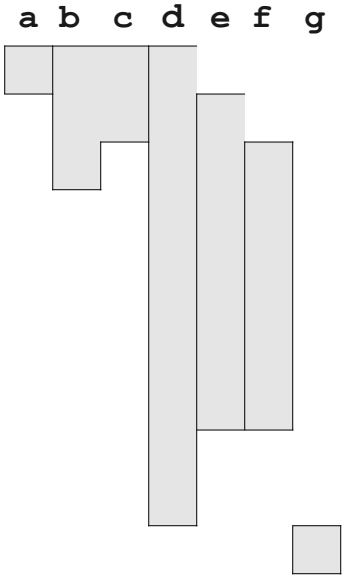
Register Allocation with Live Intervals

- Given the live intervals for all the variables in the program, we can allocate registers using a simple greedy algorithm.
- Idea: Track which registers are free at each point.
- When a live interval begins, give that variable a free register.
- When a live interval ends, the register is once again free.
- We can't always fit everything into a register; we'll see what do to in a minute.

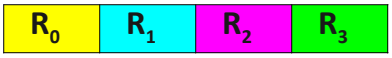


21

Register Allocation with Live Intervals

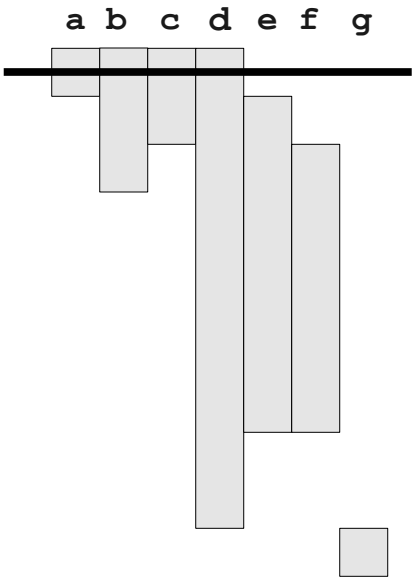


Free Registers



22

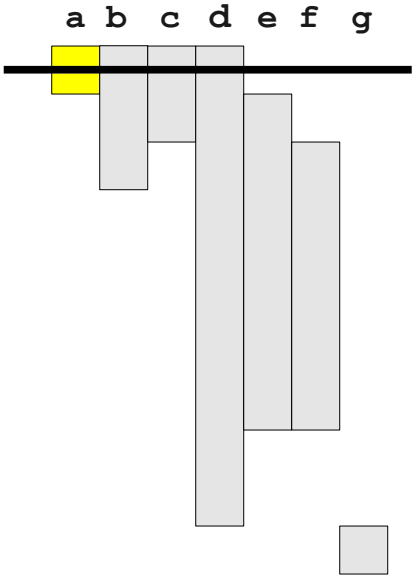
Register Allocation with Live Intervals



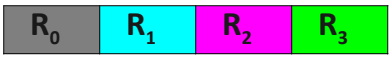
Free Registers

23

Register Allocation with Live Intervals

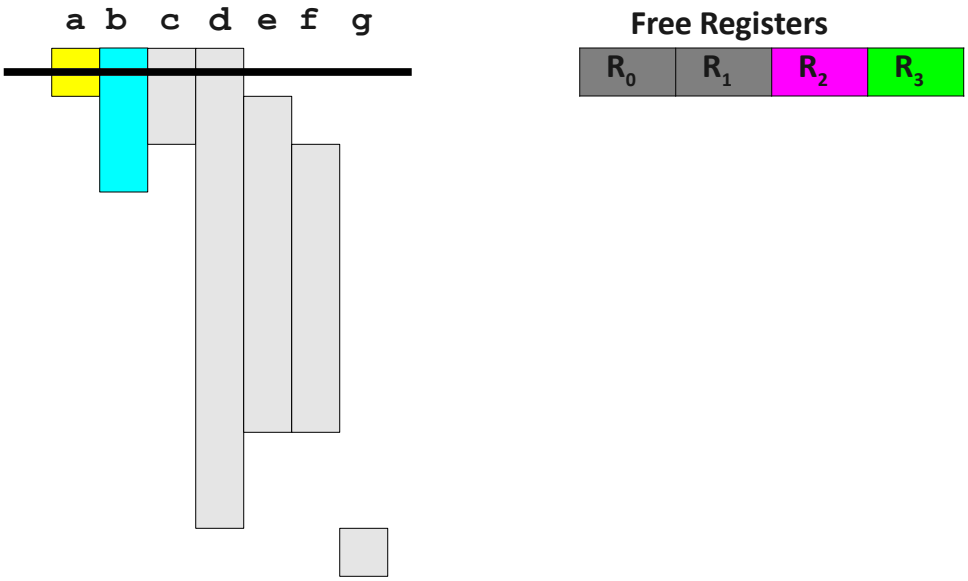


Free Registers



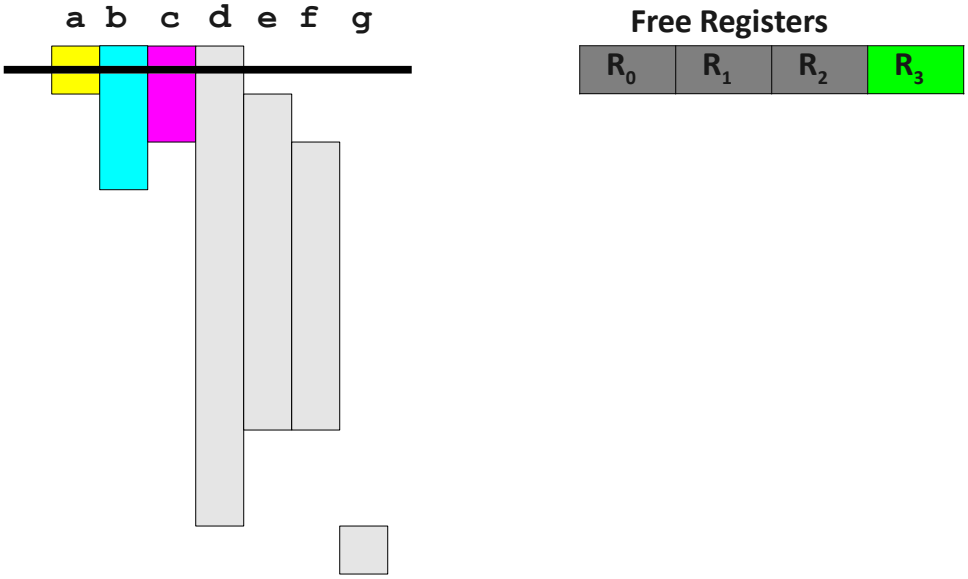
24

Register Allocation with Live Intervals



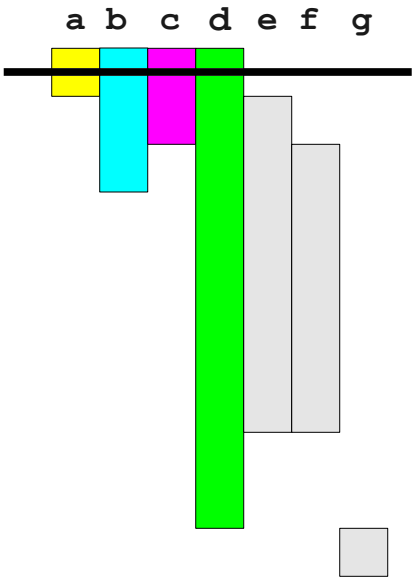
25

Register Allocation with Live Intervals



26

Register Allocation with Live Intervals

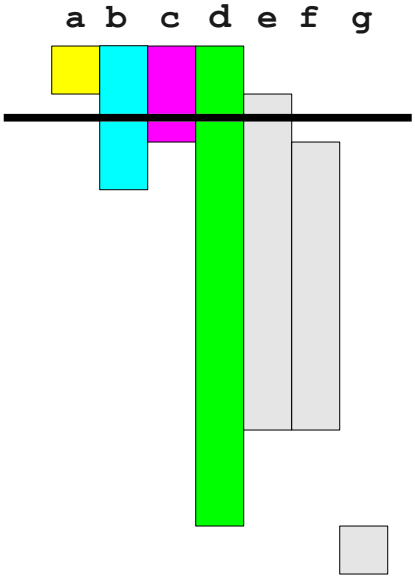


Free Registers

R_0	R_1	R_2	R_2

27

Register Allocation with Live Intervals

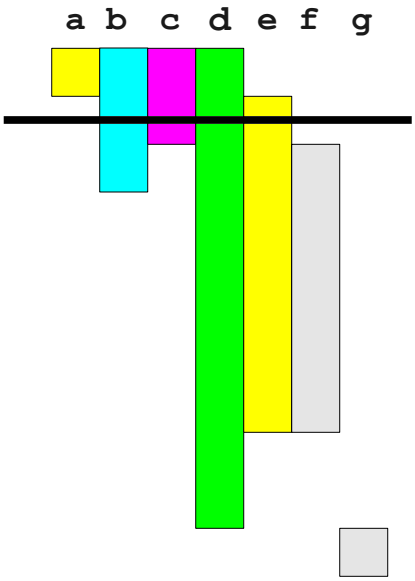


Free Registers

R_0	R_1	R_2	R_2

28

Register Allocation with Live Intervals

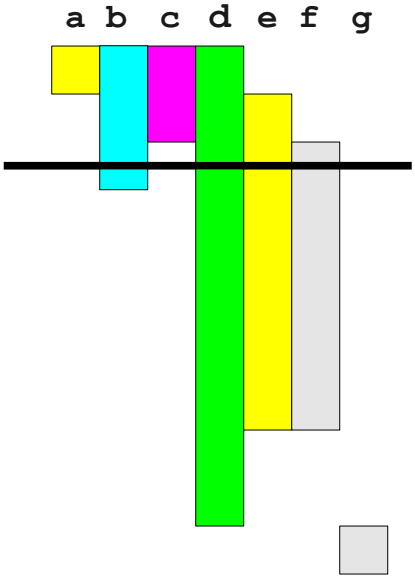


Free Registers

R_0	R_1	R_2	R_2

29

Register Allocation with Live Intervals

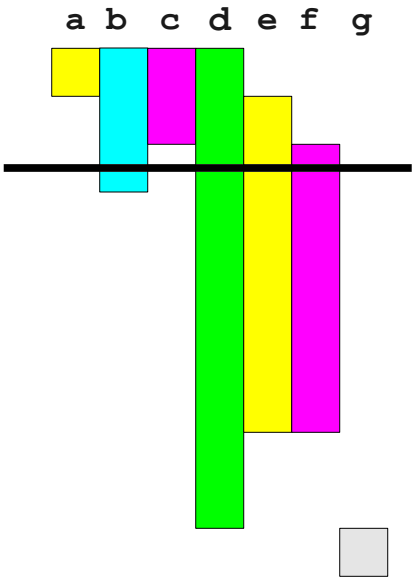


Free Registers

R_0	R_1	R_2	R_2

30

Register Allocation with Live Intervals

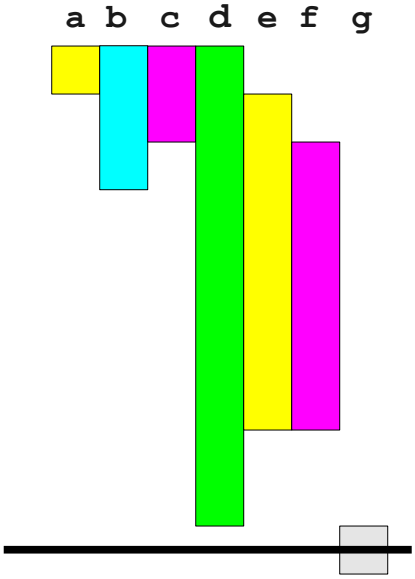


Free Registers

R_0	R_1	R_2	R_2

31

Register Allocation with Live Intervals

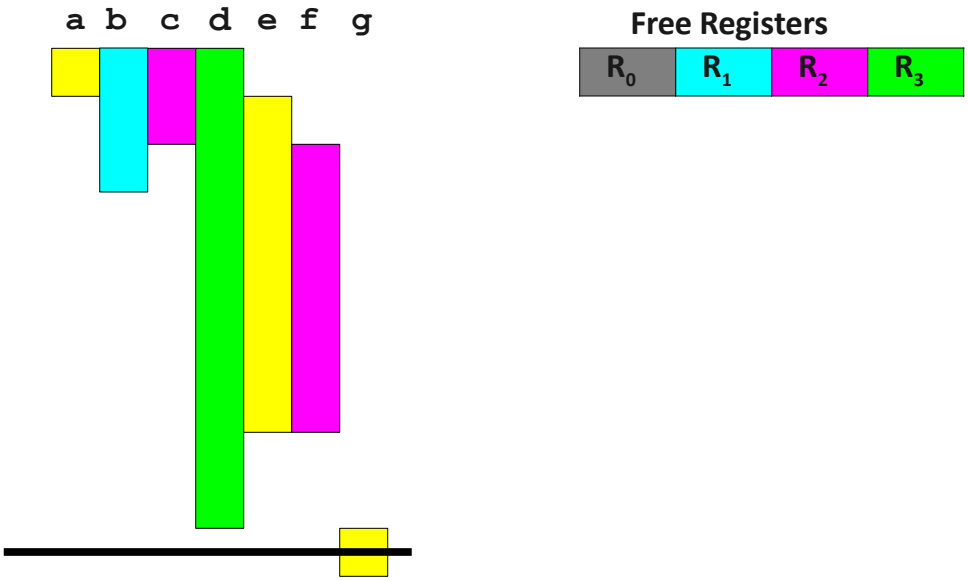


Free Registers

R_0	R_1	R_2	R_3

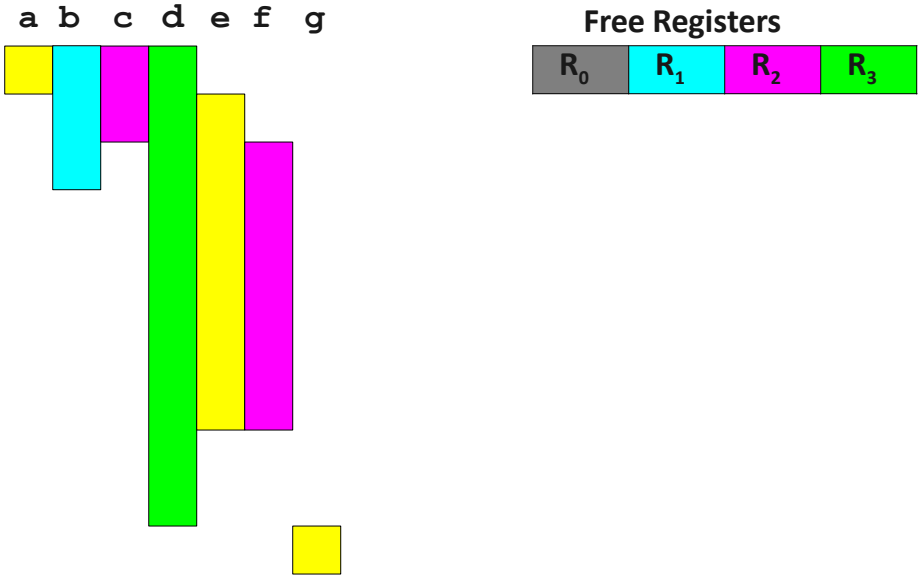
32

Register Allocation with Live Intervals



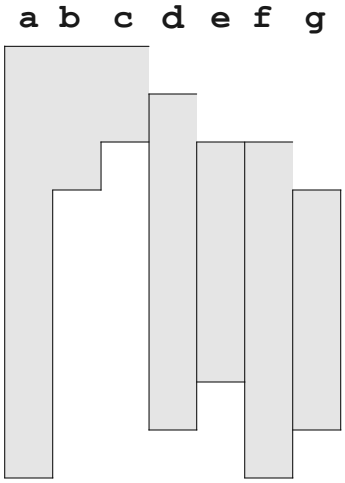
33

Register Allocation with Live Intervals



34

Another Example

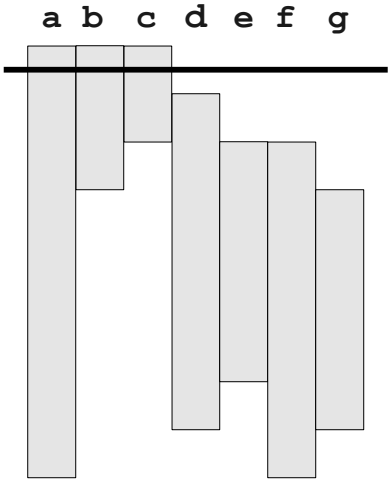


Free Registers

R_0	R_1	R_2

35

Another Example

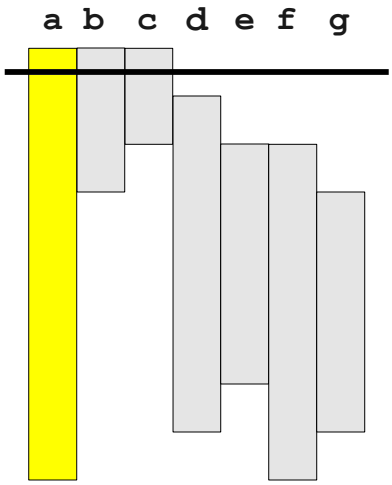


Free Registers

R_0	R_1	R_2

36

Another Example

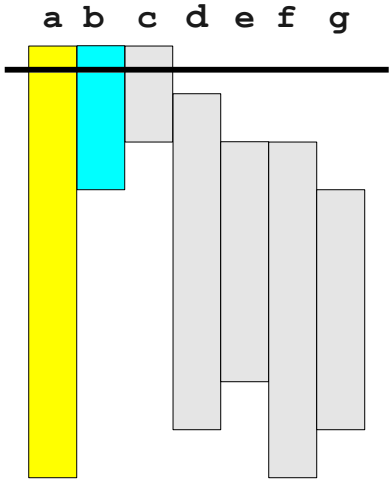


Free Registers

R_0	R_1	R_2

37

Another Example

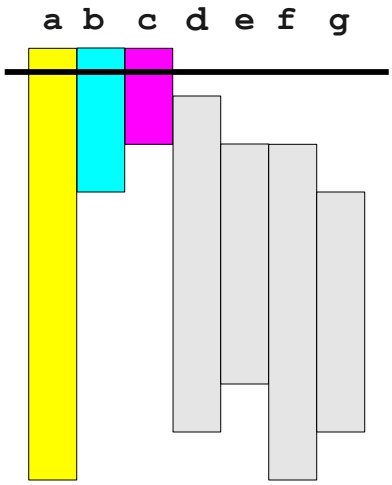


Free Registers

R_0	R_1	R_2

38

Another Example

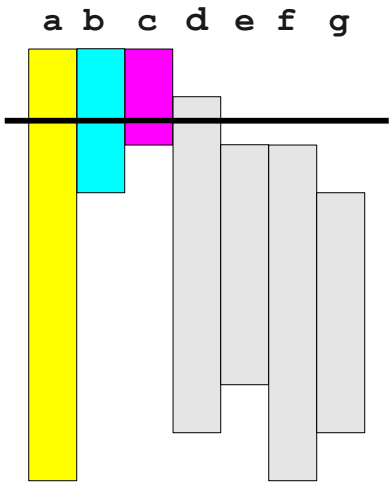


Free Registers

R_0	R_1	R_2

39

Another Example



Free Registers

R_0	R_1	R_2

What do we do now?

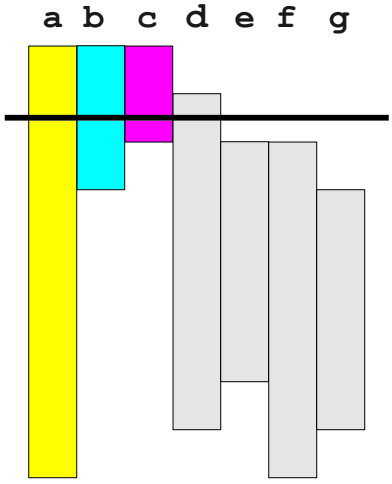
40

Register Spilling

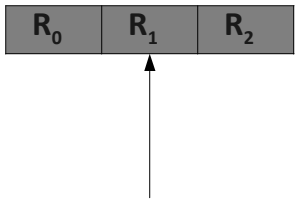
- If a register cannot be found for a variable *v*, we may need to **spill** a variable.
- When a variable is spilled, it is stored in memory rather than a register.
- When we need a register for the spilled variable:
  - Remove some variable from register to memory.
  - Load the variable into the register.
  - When done, write the register back to memory and reload the register with its original value.
- Spilling is slow, but sometimes necessary.

41

Another Example



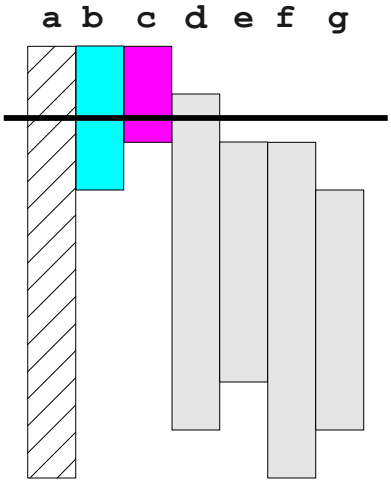
Free Registers



What do we do now?

42

Another Example

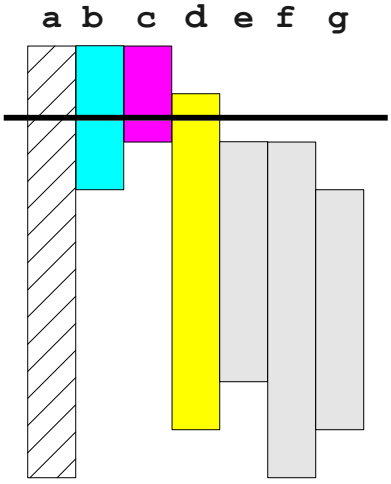


Free Registers

R_0	R_1	R_2

43

Another Example

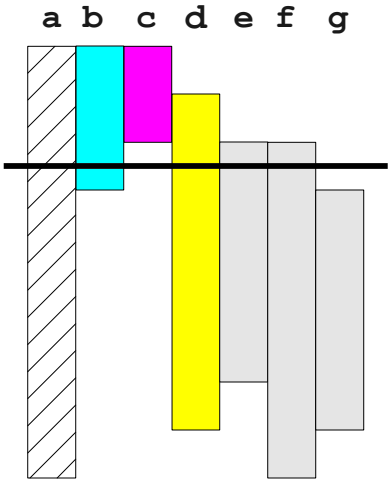


Free Registers

R_0	R_1	R_2

44

Another Example

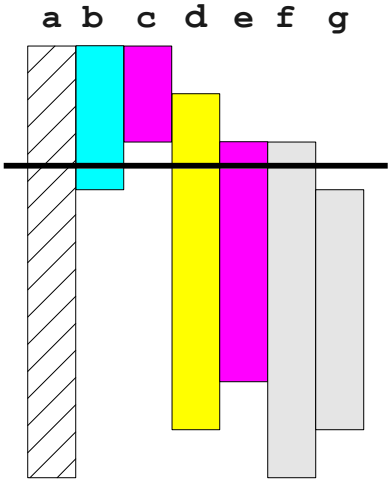


Free Registers

R_0	R_1	R_2

45

Another Example

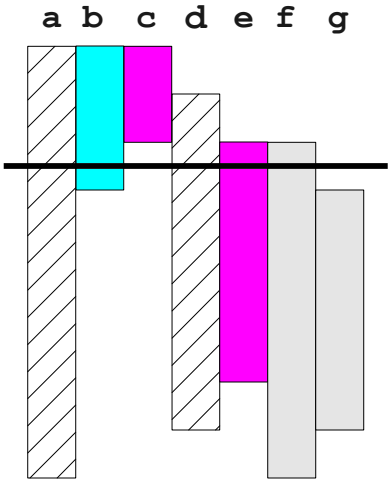


Free Registers

R_0	R_1	R_2

46

Another Example

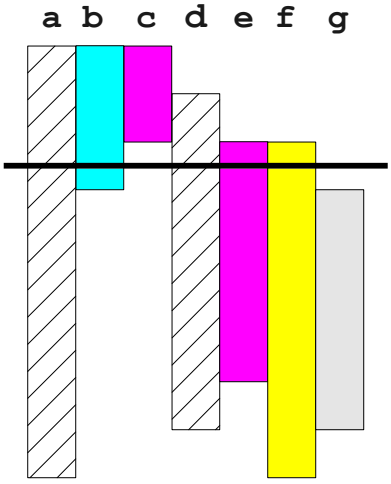


Free Registers

R_0	R_1	R_2

47

Another Example

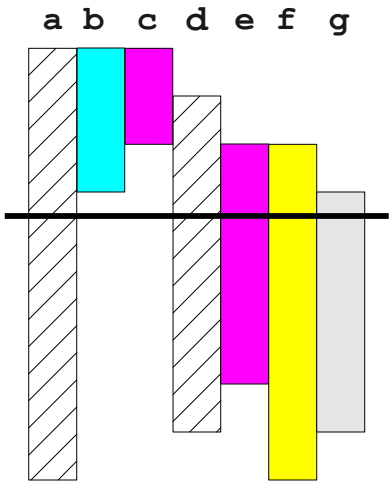


Free Registers

R_0	R_1	R_2

48

Another Example

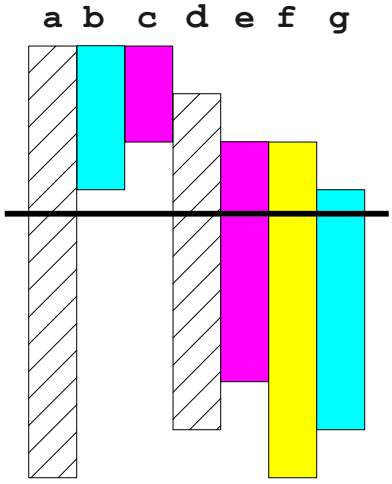


Free Registers

R_0	R_1	R_2

49

Another Example

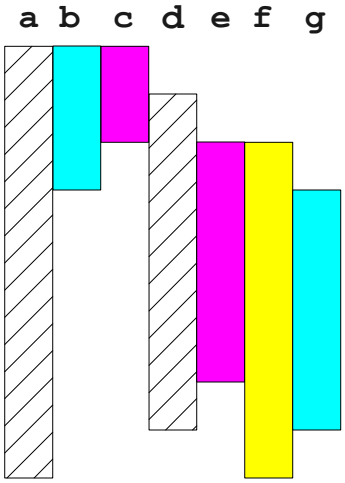


Free Registers

R_0	R_1	R_2

50

Another Example



Free Registers

R_0	R_1	R_2

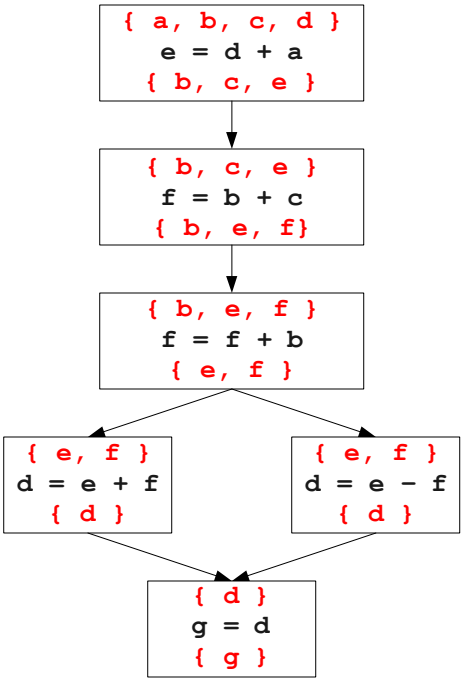
51

Linear Scan Register Allocation

- This algorithm is called **linear scan register allocation** and is a comparatively new algorithm.
- Advantages:
  - Very efficient (after computing live intervals, runs in linear time)
    Produces good code in many instances.
  - Allocation step works in one pass; can generate code during iteration.
  - Often used in JIT compilers like Java HotSpot.
  -
- Disadvantages:
  - Imprecise due to use of live **intervals** rather than live **ranges**. Other
  - techniques known to be superior in many cases.

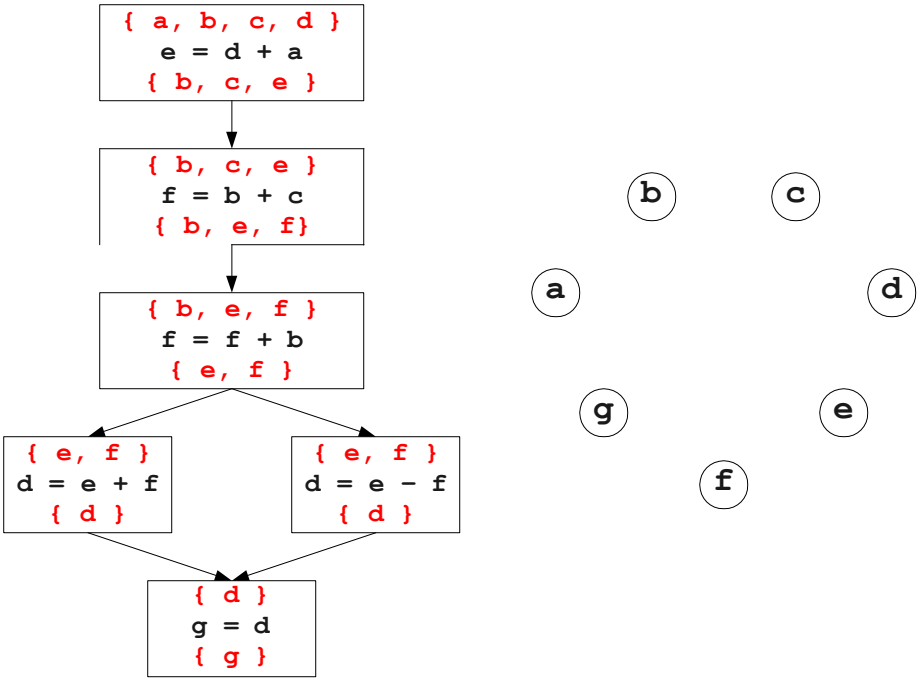
52

Register Interference Graph



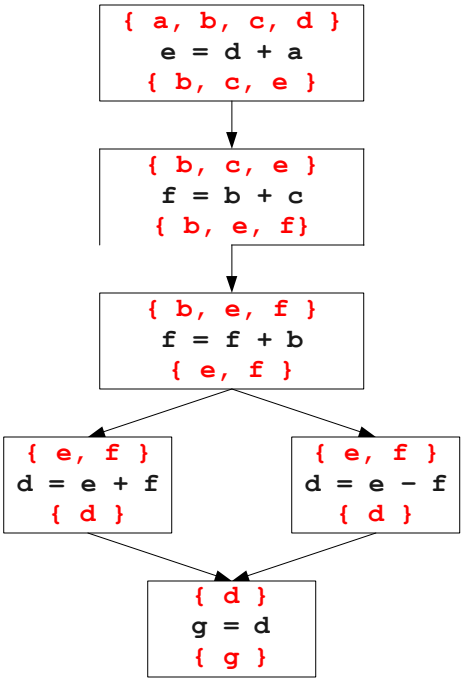
53

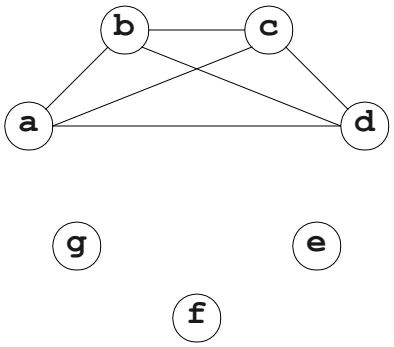
Register Interference Graph



54

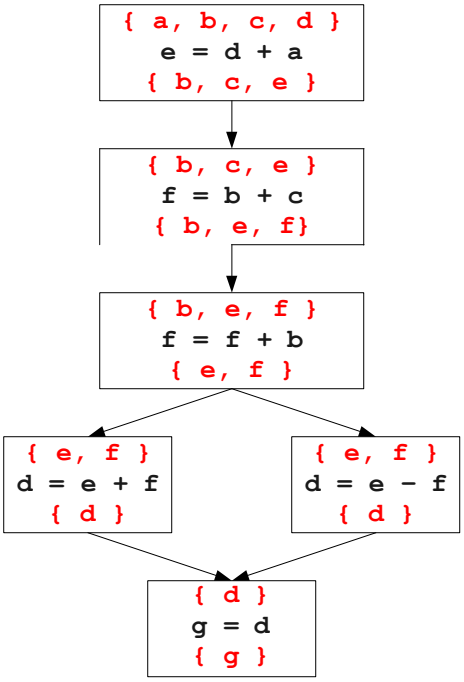
Register Interference Graph

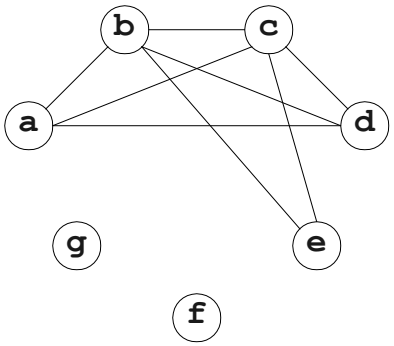




55

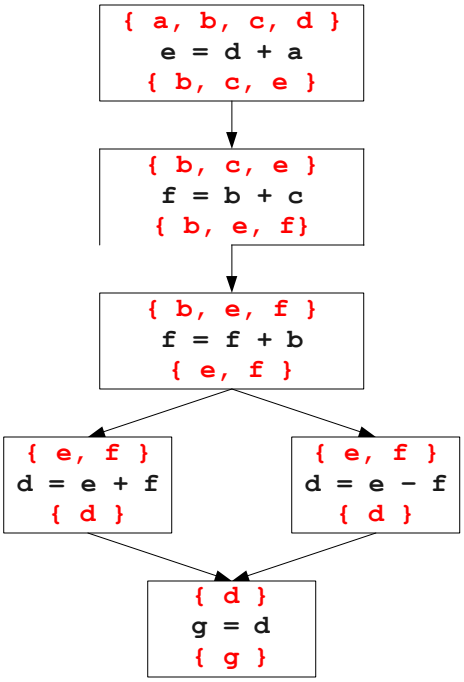
Register Interference Graph

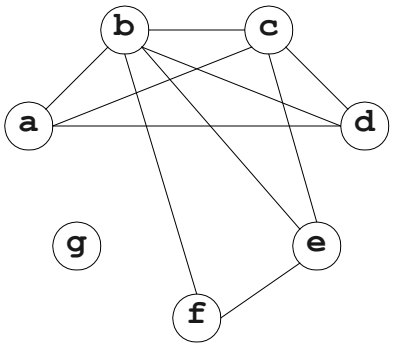




56

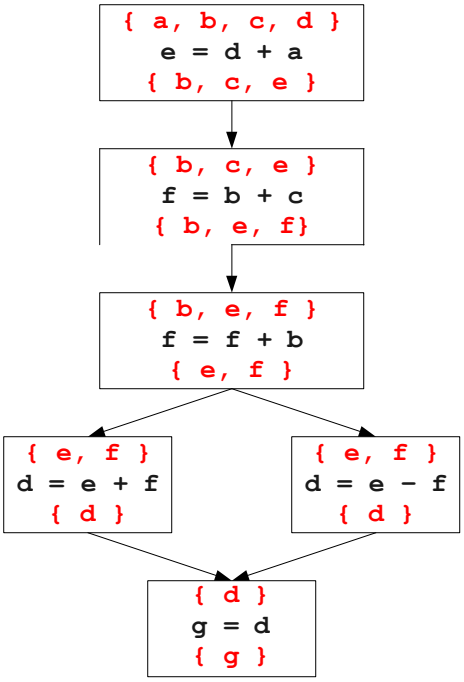
Register Interference Graph

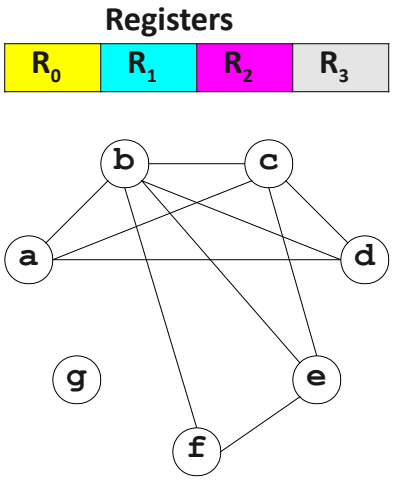




57

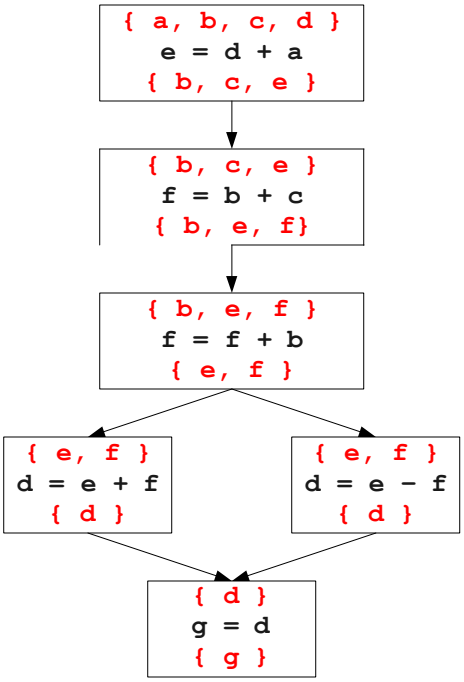
Register Interference Graph

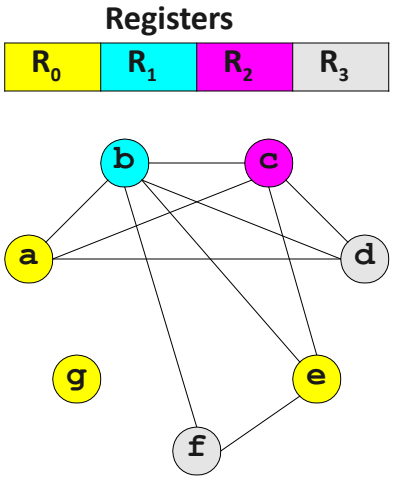




58

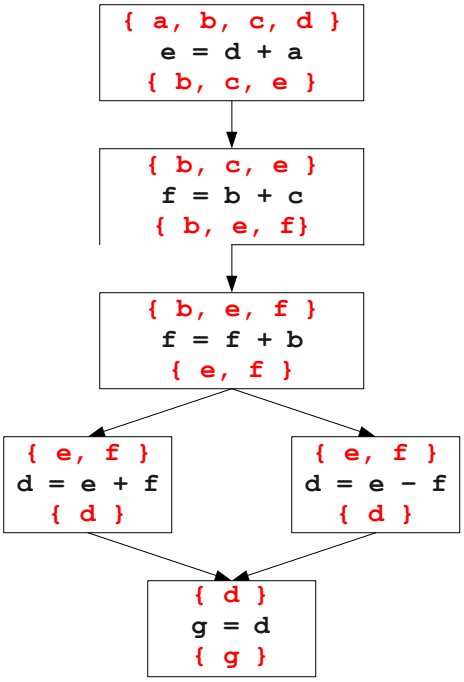
Register Interference Graph

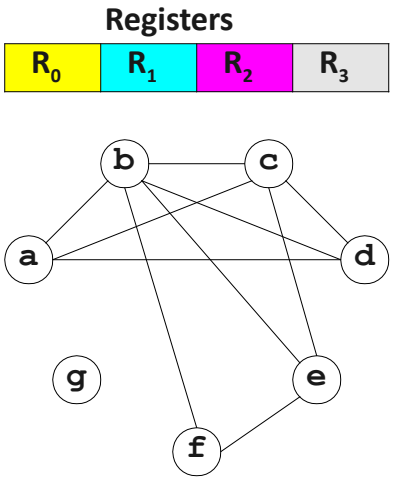




59

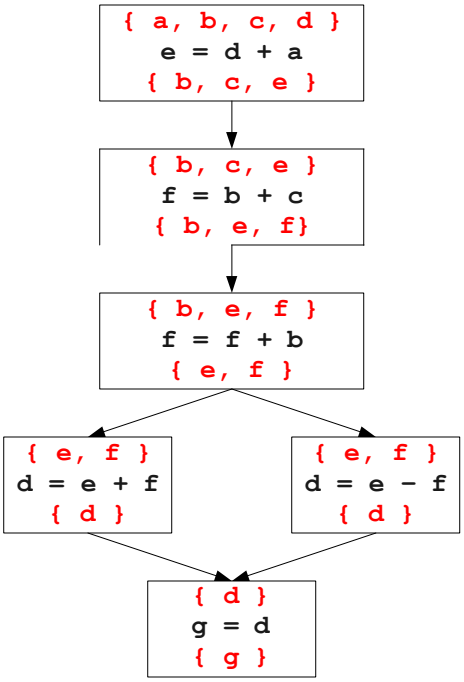
Register Interference Graph

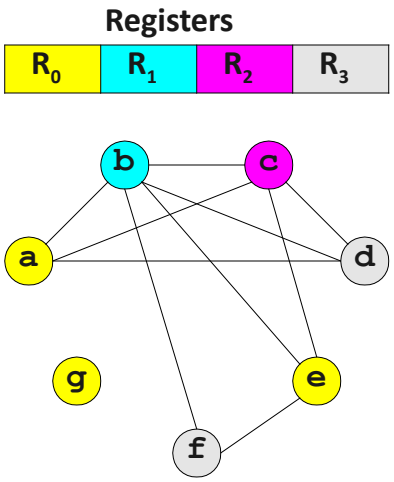




60

Register Interference Graph





61

The Register Interference Graph

- The **register interference graph** (RIG) of a control-flow graph is an undirected graph where
  - Each node is a variable.
  - There is an edge between two variables that are live at the same program point.
- Perform register allocation by assigning each variable a different register from all of its neighbors.

  There's just one catch...
-

62

The One Catch

- This problem is equivalent to **graph-coloring**, which is **NP-hard** if there are at least three registers.
- No good polynomial-time algorithms (or even good approximations!) are known for this problem.
- We have to be content with a heuristic that is good enough for RIGs that arise in practice.

63

Chaitin's Algorithm

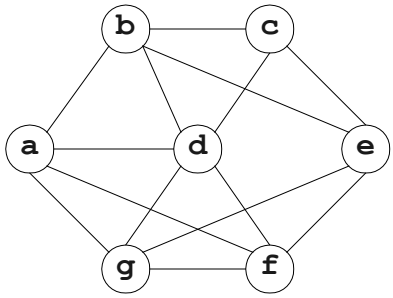
- Intuition:
  - Suppose we are trying to *k*-color a graph and find a node with fewer than *k* edges.
  - If we delete this node from the graph and color what remains, we can find a color for this node if we add it back in.

    Reason: With fewer than *k* neighbors, some color must be left over.

- Algorithm:
  - Find a node with fewer than *k* outgoing edges. Remove it
  - from the graph.
  - Recursively color the rest of the graph. Add the
  - node back in.
  - Assign it a valid color.

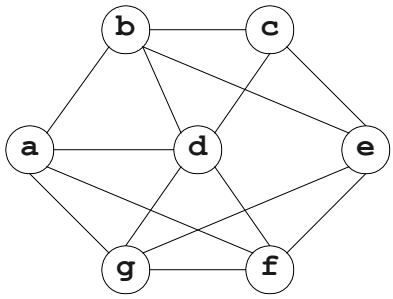
64

Chaitin's Algorithm



65

Chaitin's Algorithm

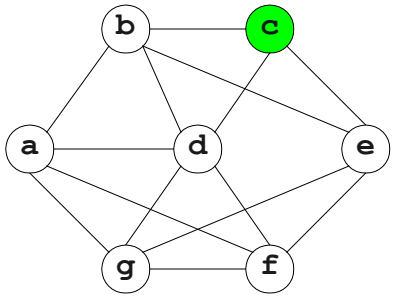


Registers

R_0	R_1	R_2	R_3

66

Chaitin's Algorithm

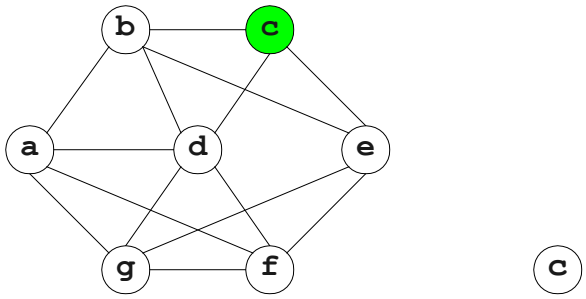


Registers

R_0	R_1	R_2	R_3

67

Chaitin's Algorithm

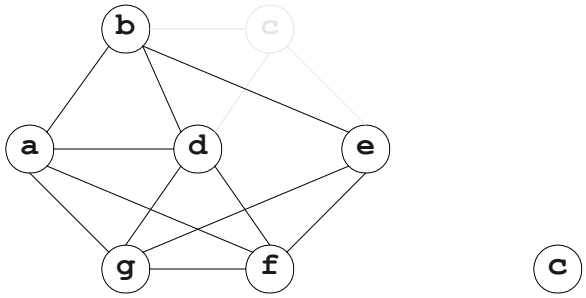


Registers

R_0	R_1	R_2	R_3

68

Chaitin's Algorithm

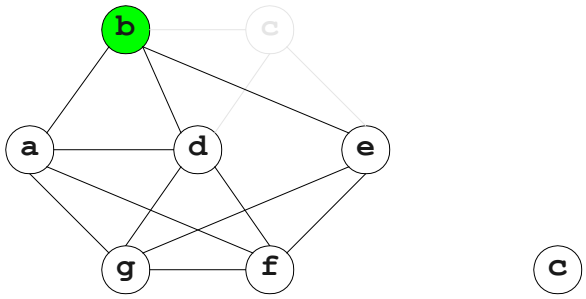


Registers

R_0	R_1	R_2	R_3

69

Chaitin's Algorithm

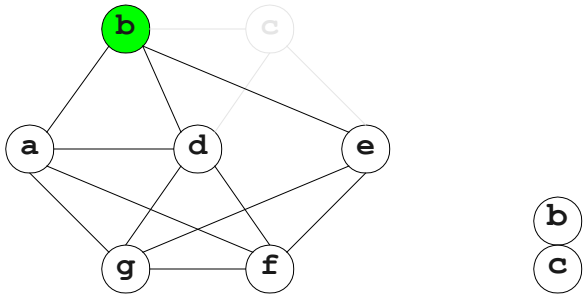


Registers

R_0	R_1	R_2	R_3

70

Chaitin's Algorithm

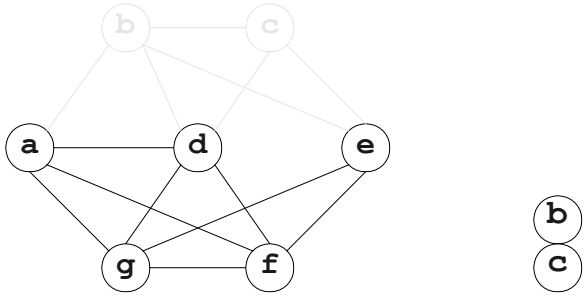


Registers

R_0	R_1	R_2	R_3

71

Chaitin's Algorithm

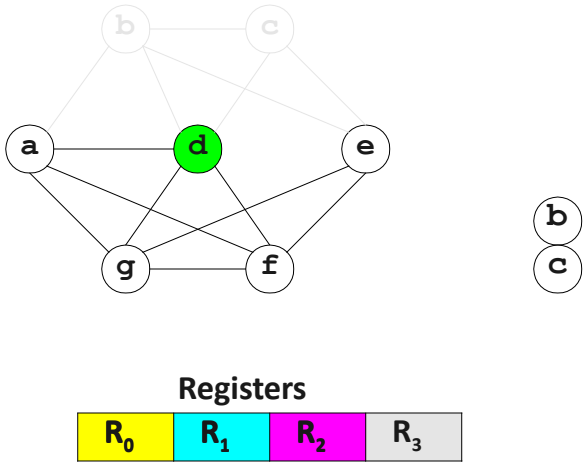


Registers

R_0	R_1	R_2	R_3

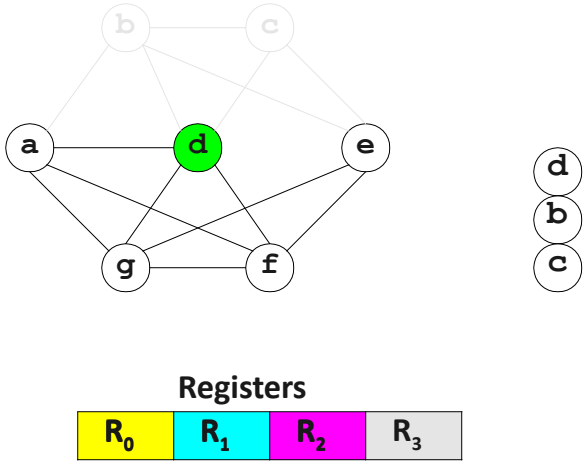
72

Chaitin's Algorithm



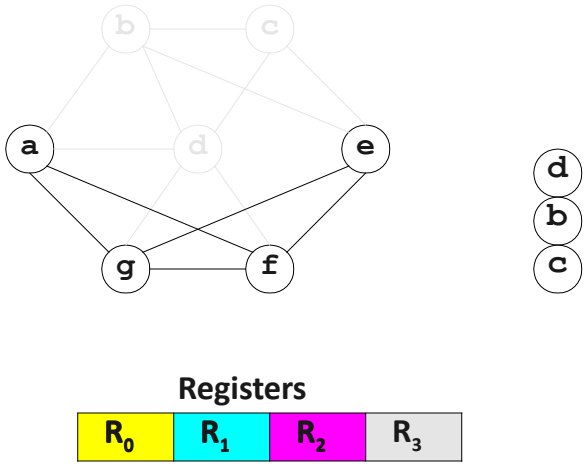
73

Chaitin's Algorithm



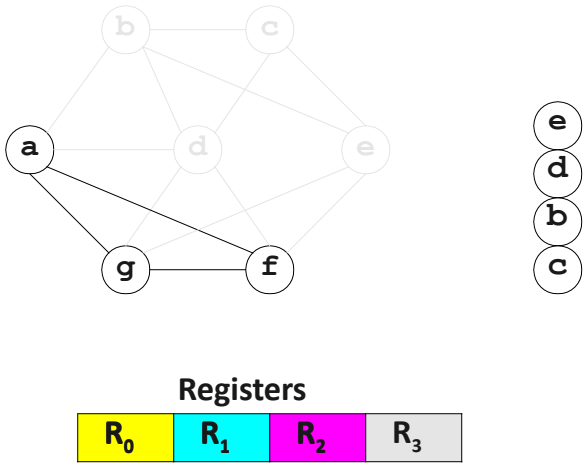
74

Chaitin's Algorithm



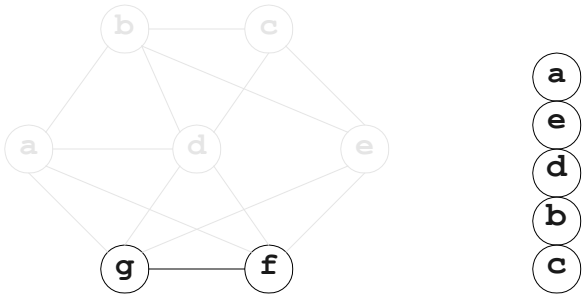
75

Chaitin's Algorithm



76

Chaitin's Algorithm

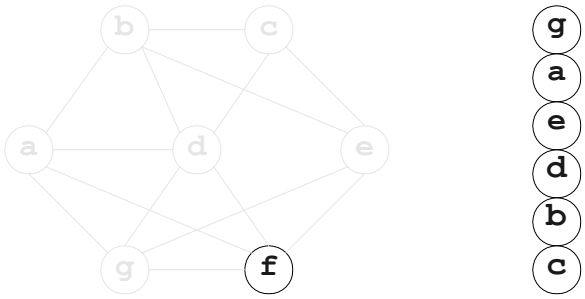


Registers

R_0	R_1	R_2	R_3

77

Chaitin's Algorithm

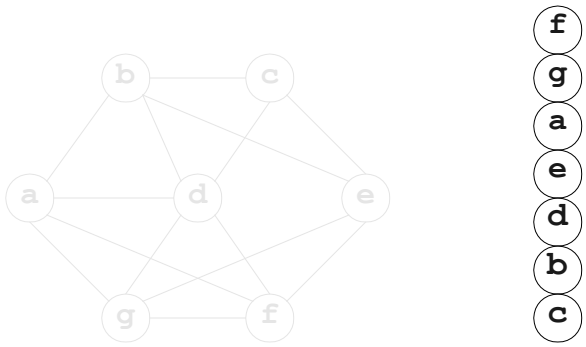


Registers

R_0	R_1	R_2	R_3

78

Chaitin's Algorithm

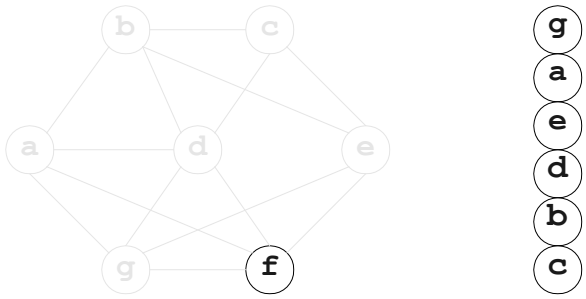


Registers

R_0	R_1	R_2	R_3

79

Chaitin's Algorithm

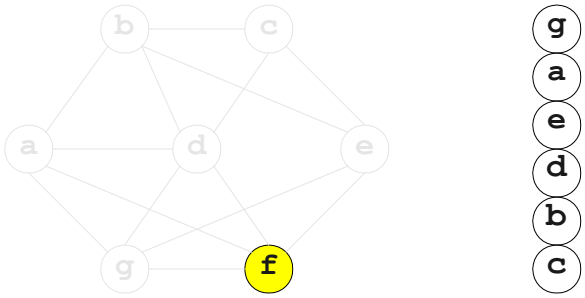


Registers

R_0	R_1	R_2	R_3

80

Chaitin's Algorithm

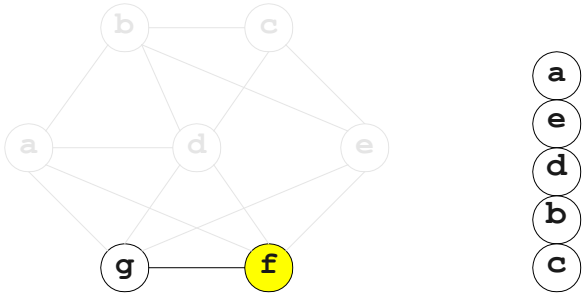


Registers

R_0	R_1	R_2	R_3

81

Chaitin's Algorithm

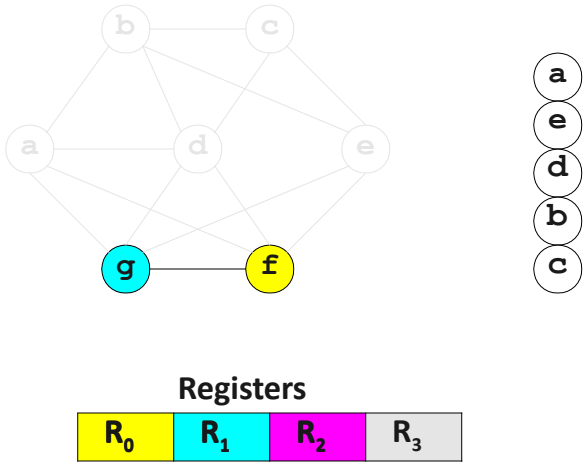


Registers

R_0	R_1	R_2	R_3

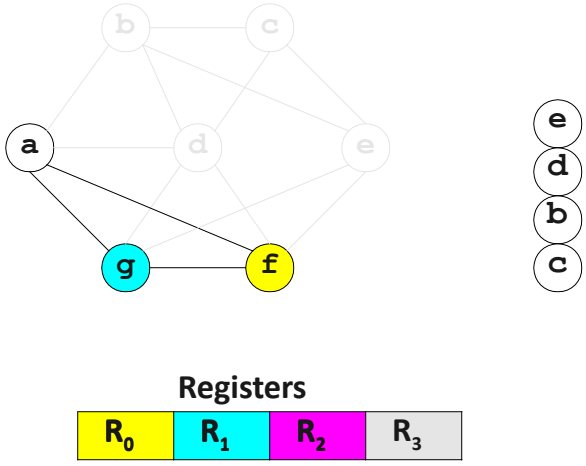
82

Chaitin's Algorithm



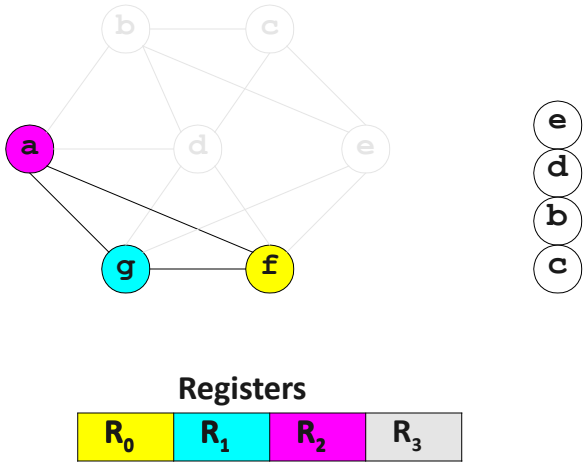
83

Chaitin's Algorithm



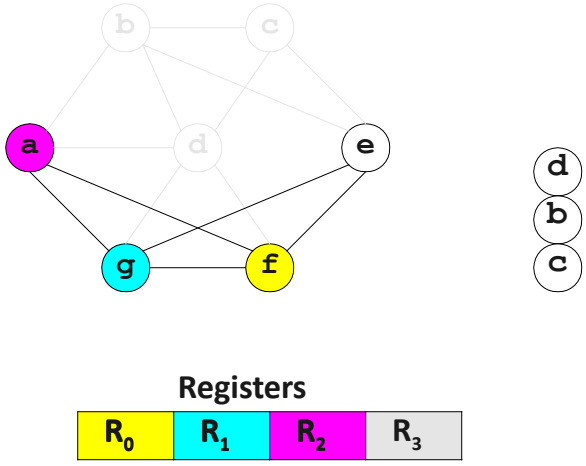
84

Chaitin's Algorithm



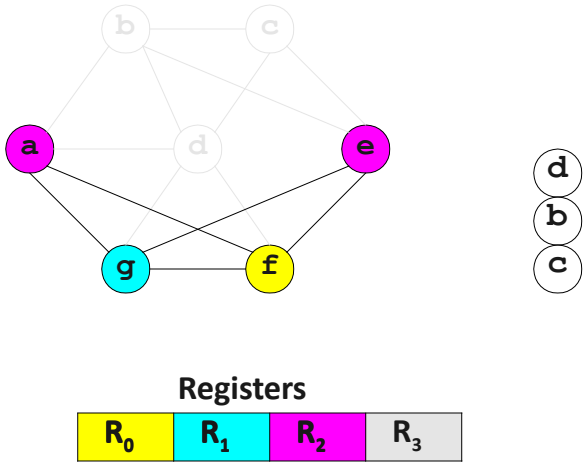
85

Chaitin's Algorithm



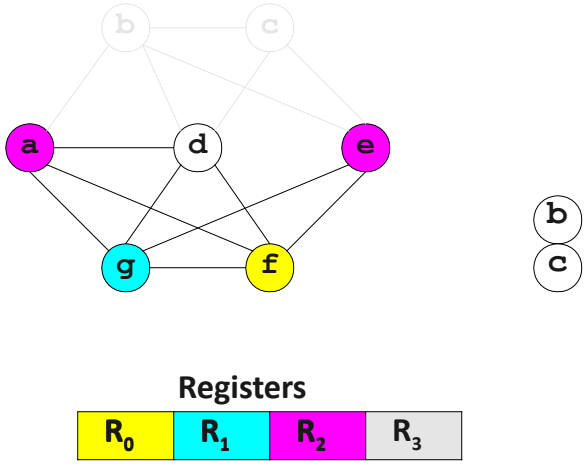
86

Chaitin's Algorithm



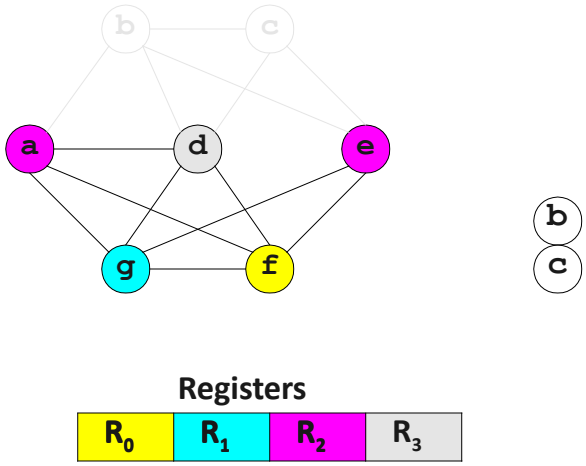
87

Chaitin's Algorithm



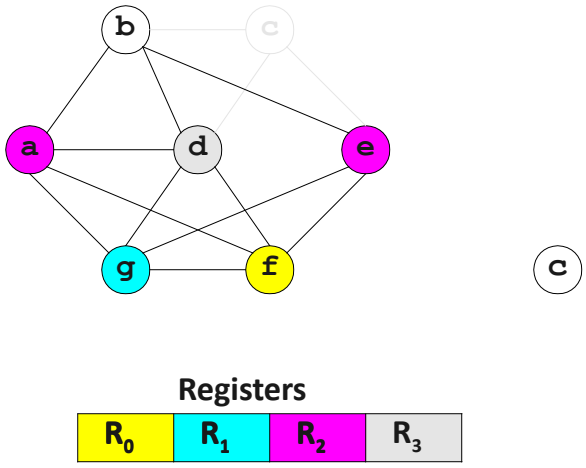
88

Chaitin's Algorithm



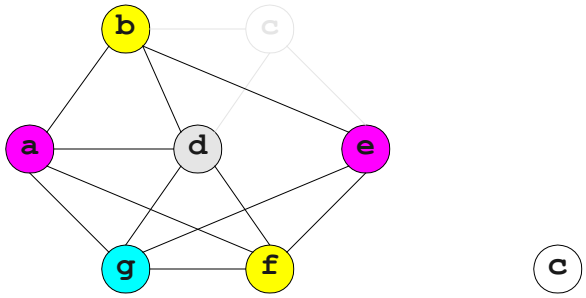
89

Chaitin's Algorithm



90

Chaitin's Algorithm

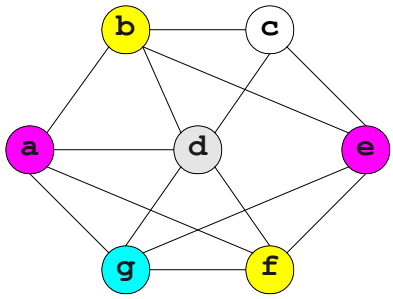


Registers

R_0	R_1	R_2	R_3

91

Chaitin's Algorithm

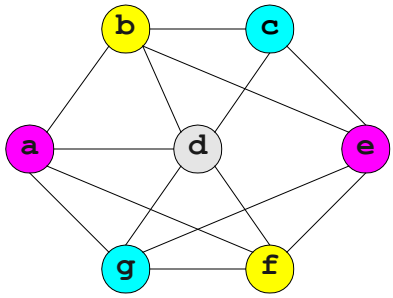


Registers

R_0	R_1	R_2	R_3

92

Chaitin's Algorithm



Registers

R_0	R_1	R_2	R_3

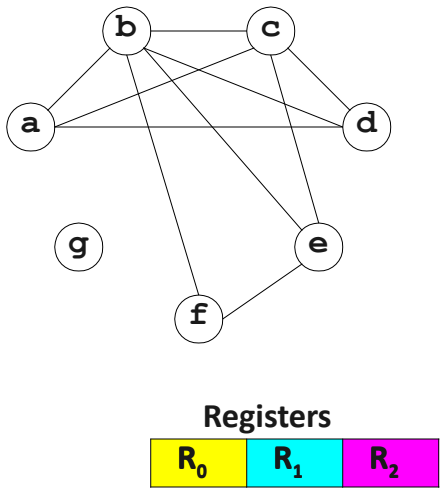
93

One Problem

- What if we can't find a node with fewer than *k* neighbors?
- Choose and remove an arbitrary node, marking it "troublesome."
  - Use heuristics to choose which one.
- When adding node back in, it may be possible to find a valid color.
- Otherwise, we have to spill that node.

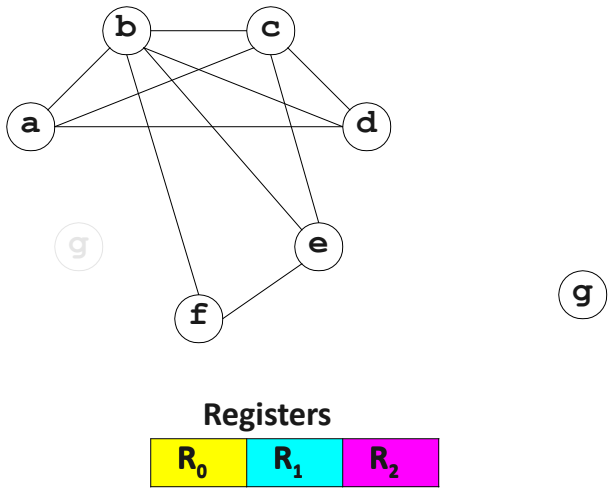
94

Chaitin's Algorithm Reloaded



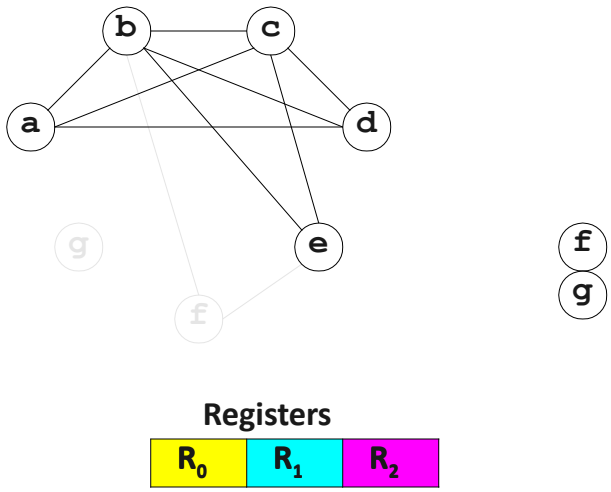
95

Chaitin's Algorithm Reloaded



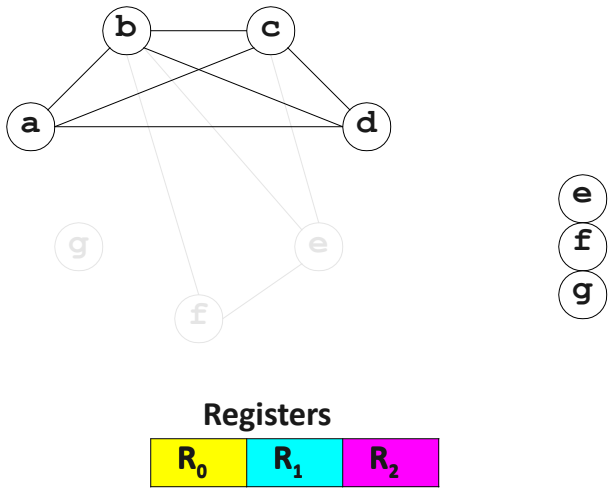
96

Chaitin's Algorithm Reloaded



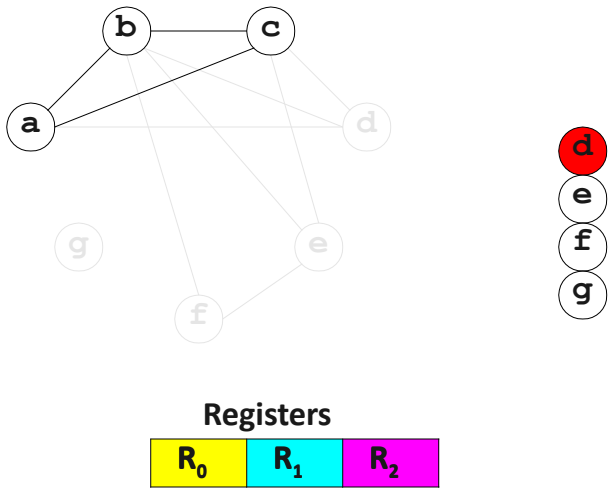
97

Chaitin's Algorithm Reloaded



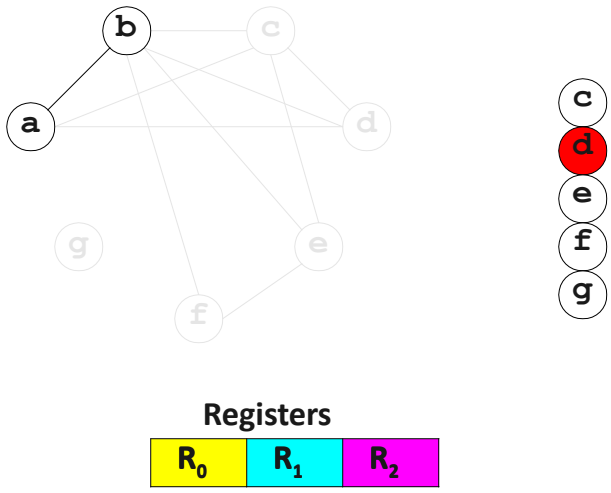
98

Chaitin's Algorithm Reloaded



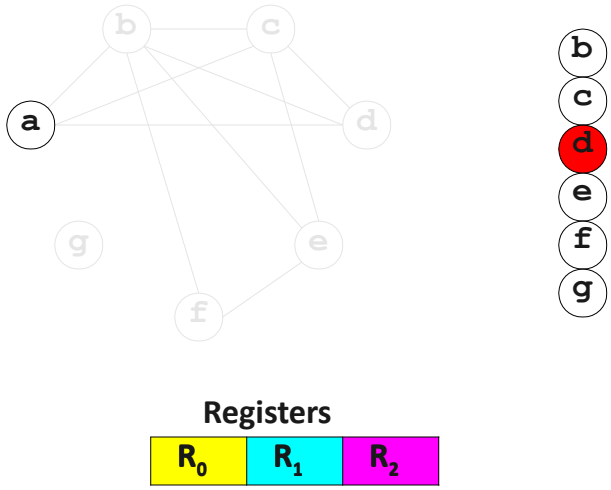
99

Chaitin's Algorithm Reloaded



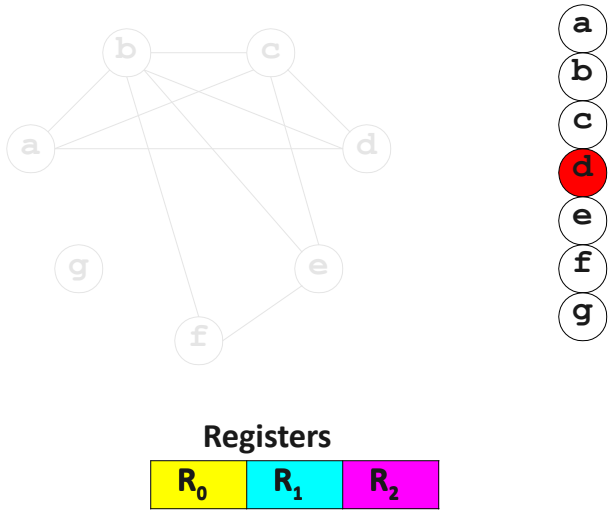
100

Chaitin's Algorithm Reloaded



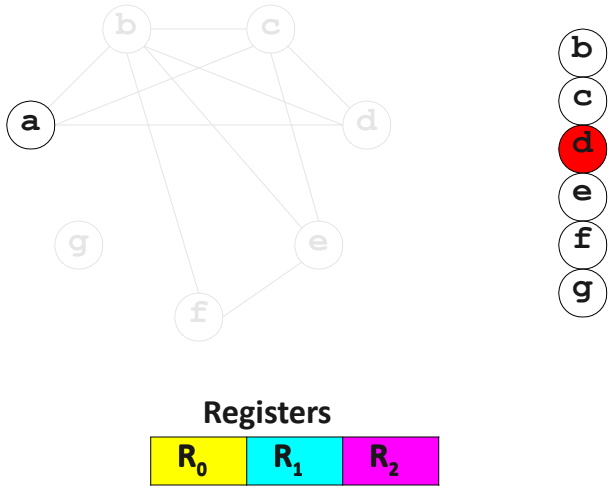
101

Chaitin's Algorithm Reloaded



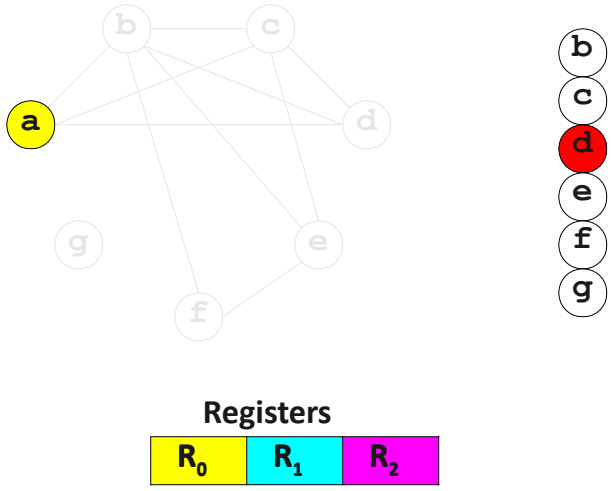
102

Chaitin's Algorithm Reloaded



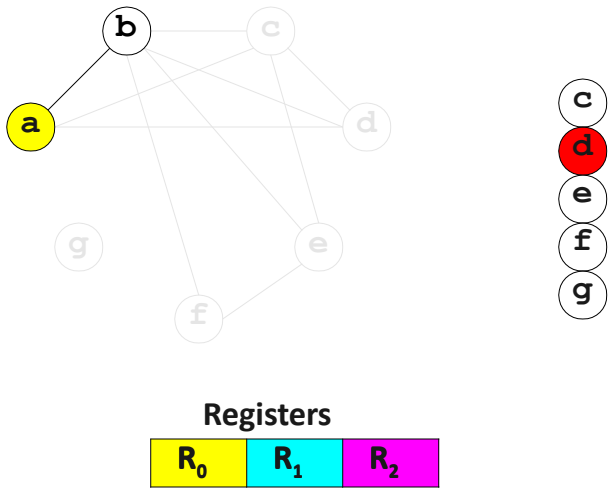
103

Chaitin's Algorithm Reloaded



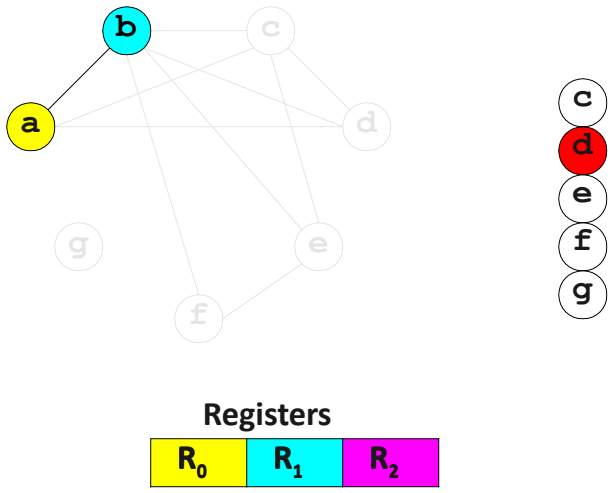
104

Chaitin's Algorithm Reloaded



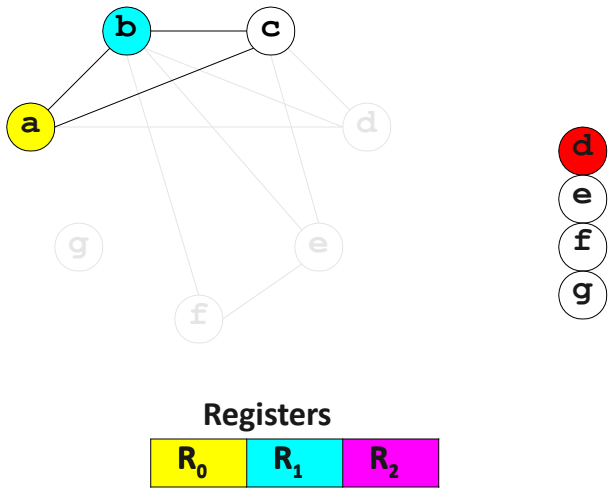
105

Chaitin's Algorithm Reloaded



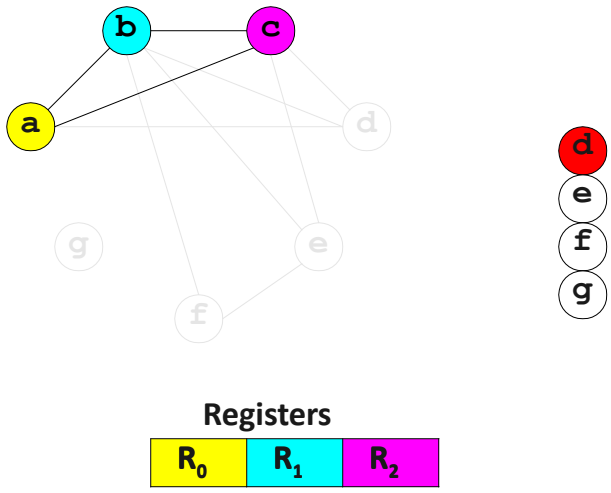
106

Chaitin's Algorithm Reloaded



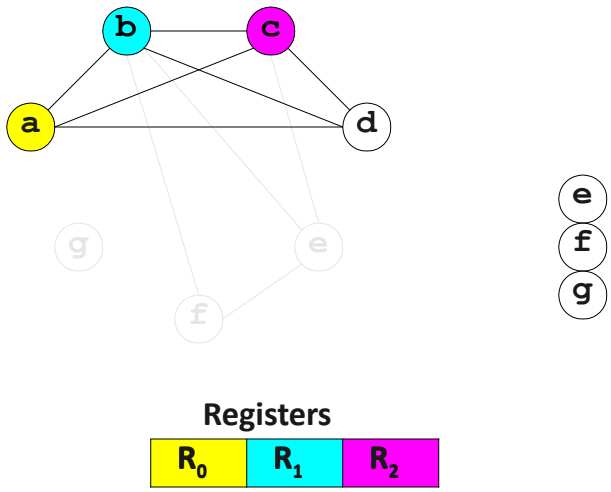
107

Chaitin's Algorithm Reloaded



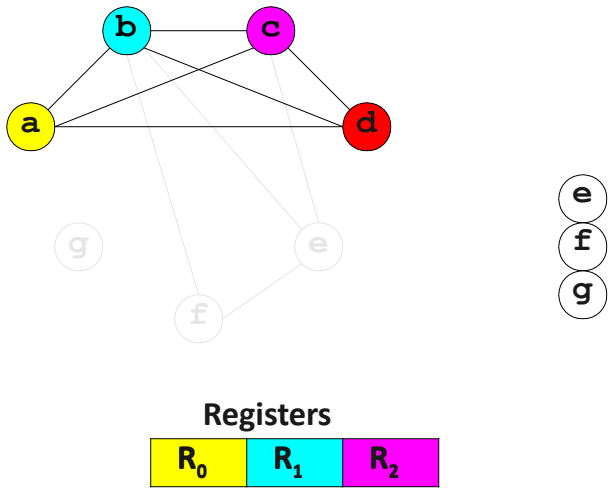
108

Chaitin's Algorithm Reloaded



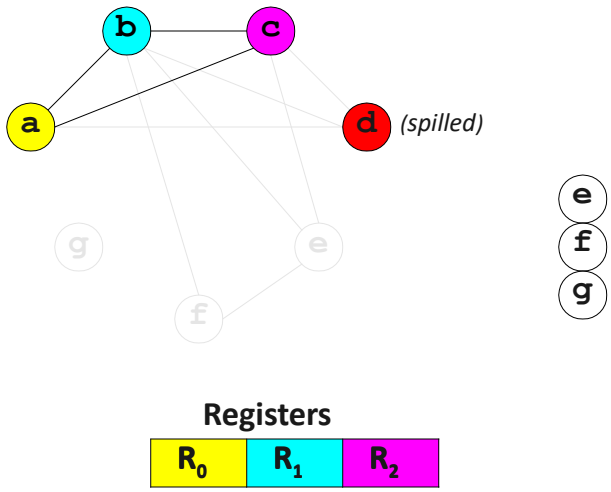
109

Chaitin's Algorithm Reloaded



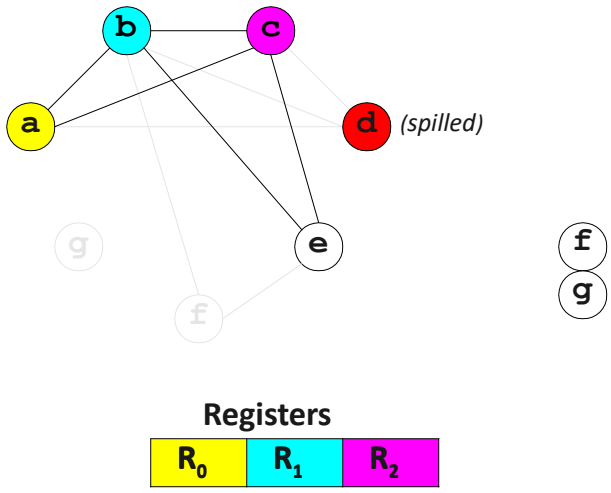
110

Chaitin's Algorithm Reloaded



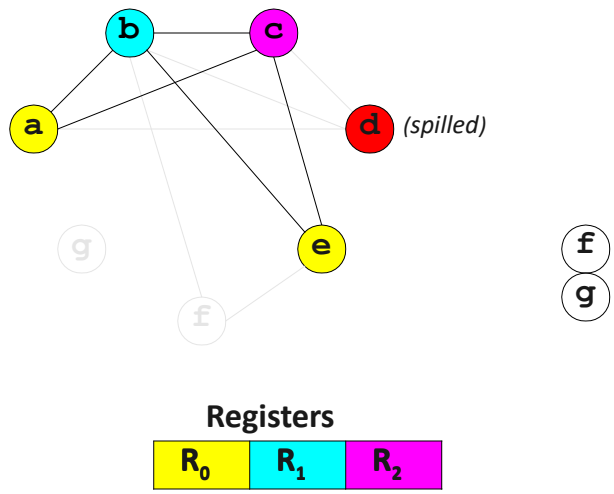
111

Chaitin's Algorithm Reloaded



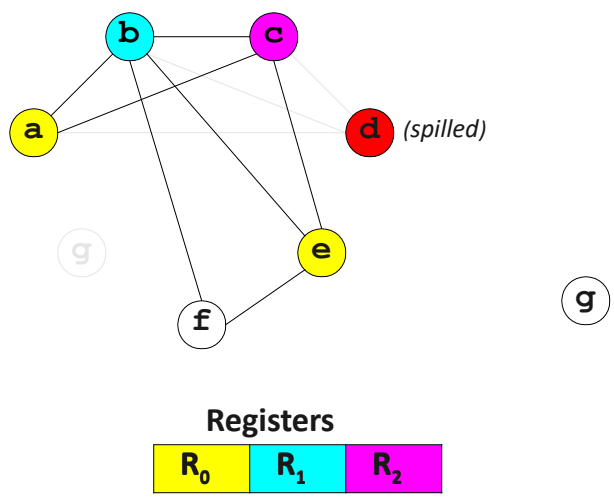
112

Chaitin's Algorithm Reloaded



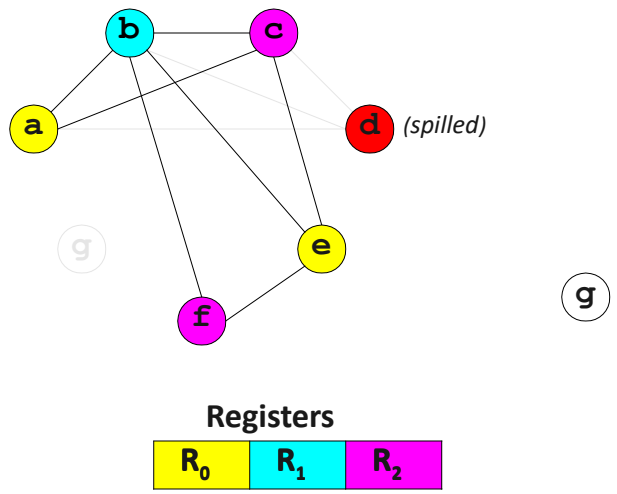
113

Chaitin's Algorithm Reloaded



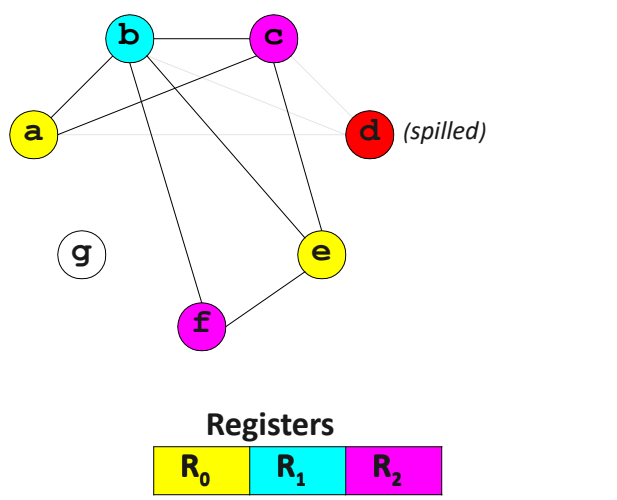
114

Chaitin's Algorithm Reloaded



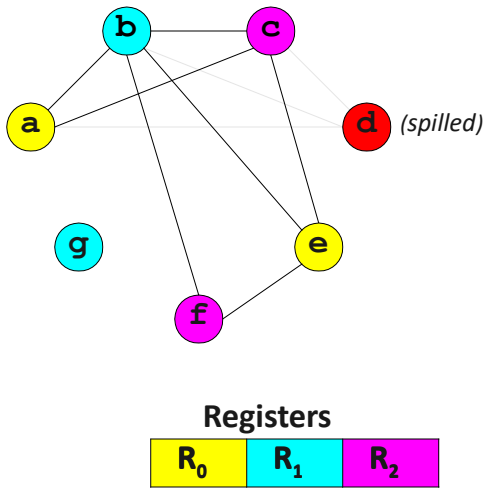
115

Chaitin's Algorithm Reloaded



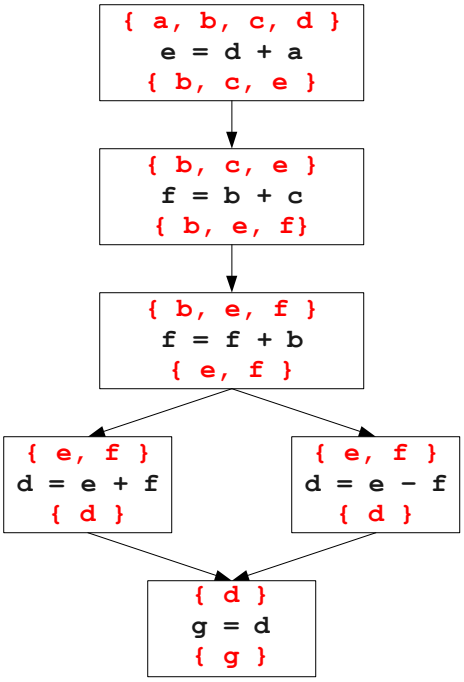
116

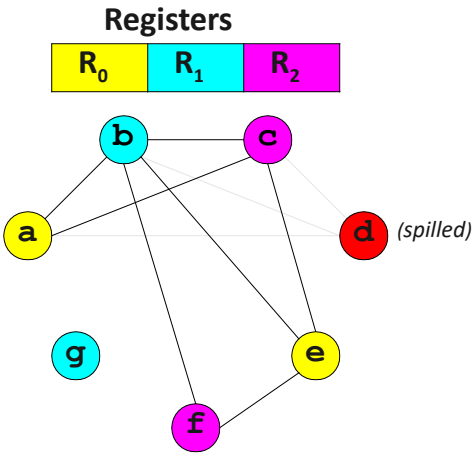
Chaitin's Algorithm Reloaded



117

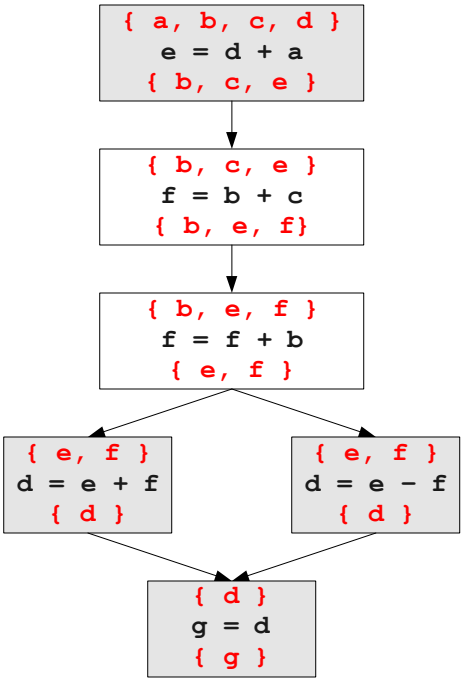
A Smarter Algorithm

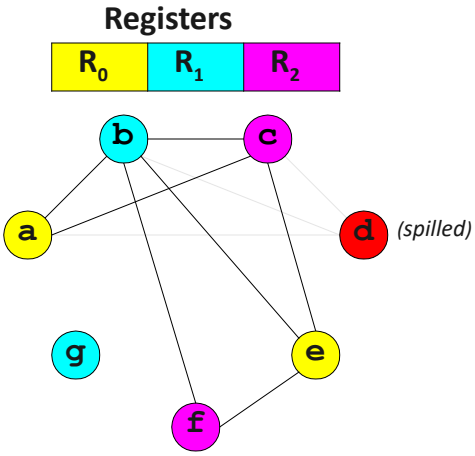




118

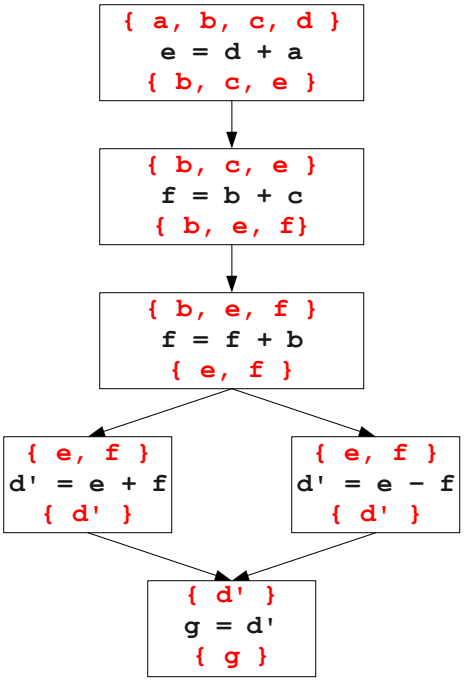
A Smarter Algorithm

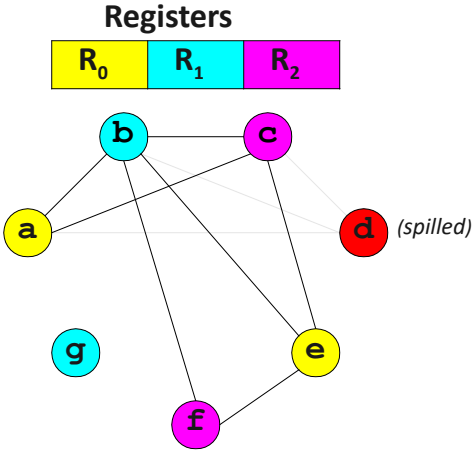




119

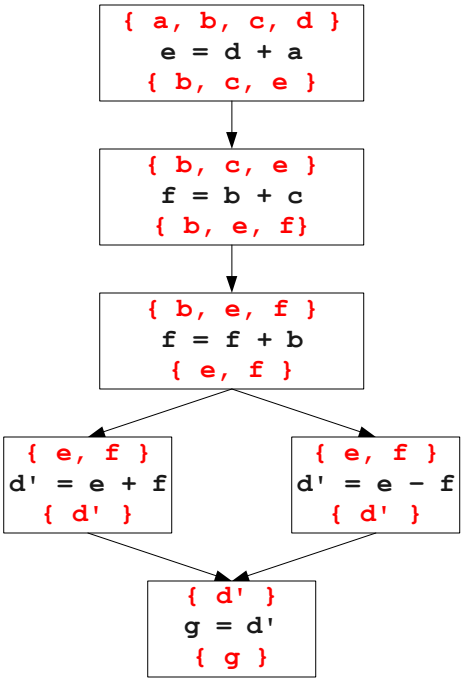
A Smarter Algorithm

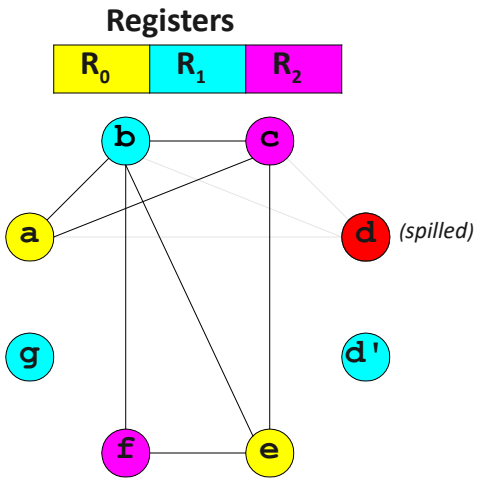




120

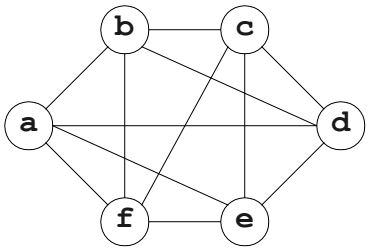
A Smarter Algorithm





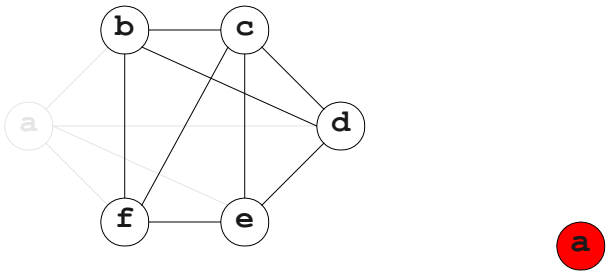
121

Another Example



122

Another Example

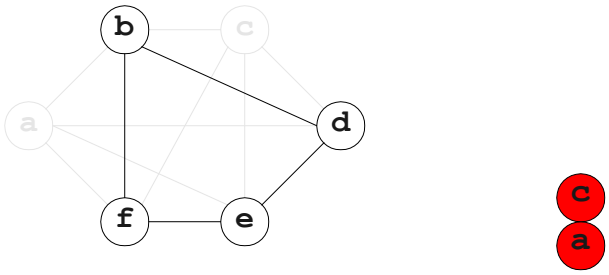


Registers

R_0	R_1	R_2

123

Another Example

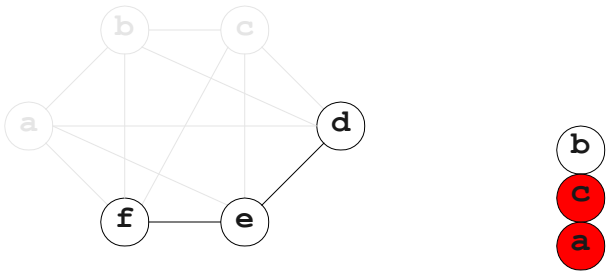


Registers

R_0	R_1	R_2

124

Another Example

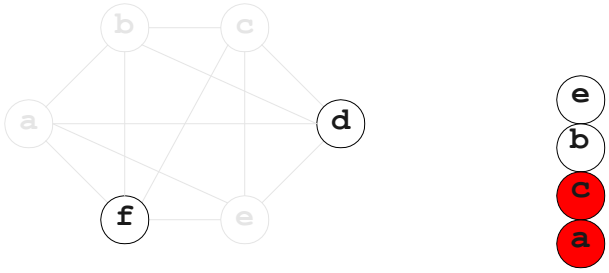


Registers

R_0	R_1	R_2

125

Another Example

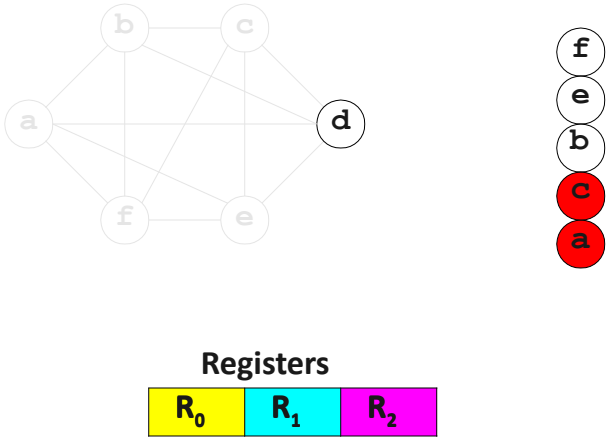


Registers

R_0	R_1	R_2

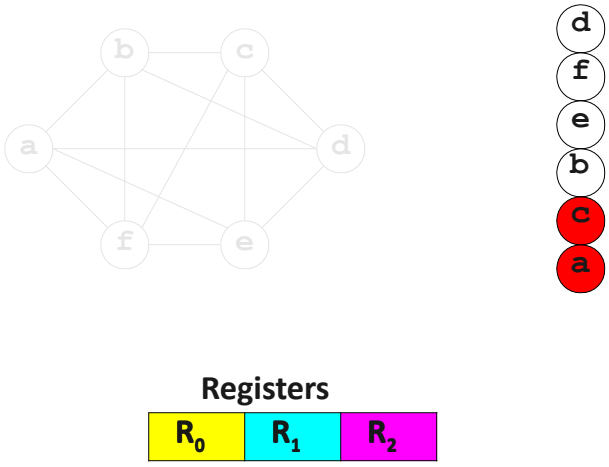
126

Another Example



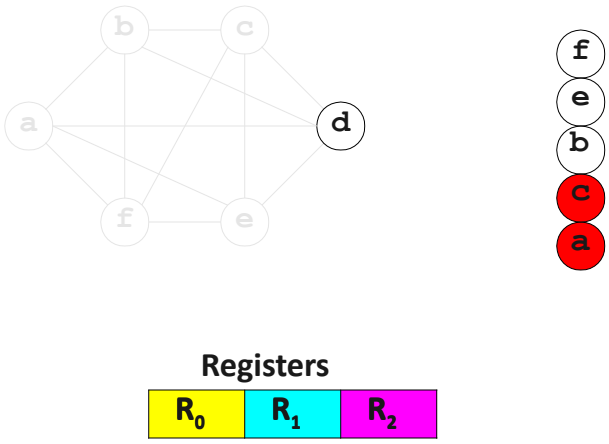
127

Another Example



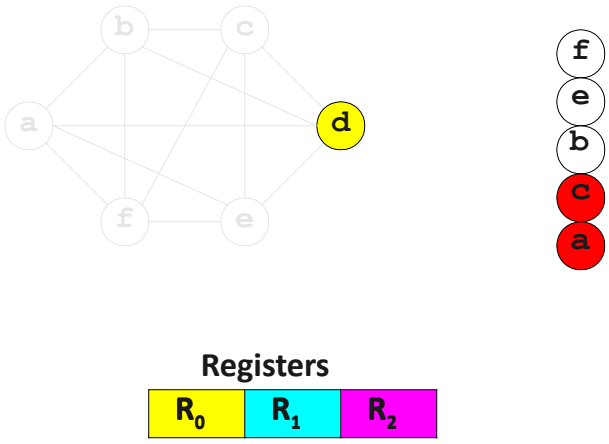
128

Another Example



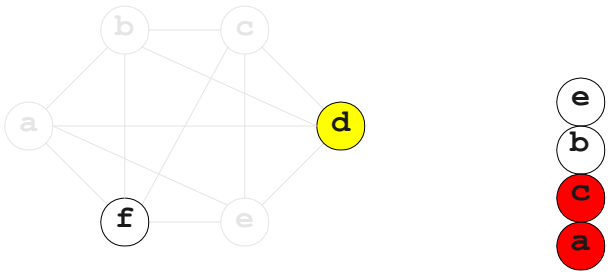
129

Another Example



130

Another Example

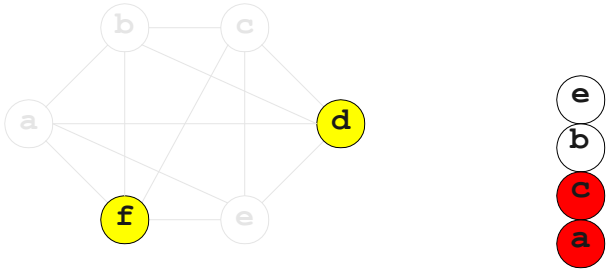


Registers

R_0	R_1	R_2

131

Another Example

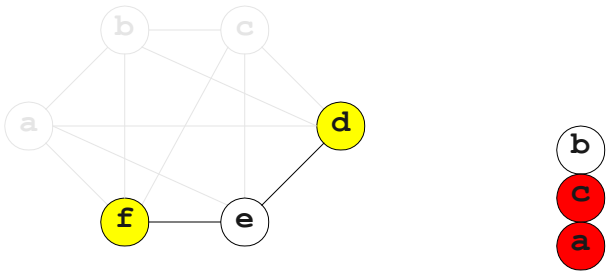


Registers

R_0	R_1	R_2

132

Another Example

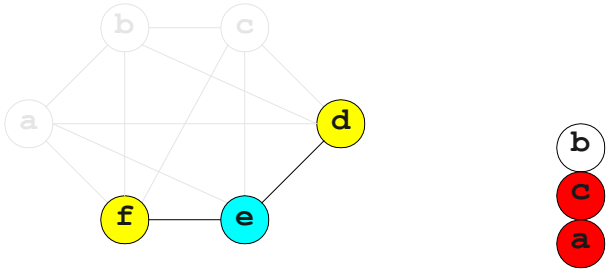


Registers

R_0	R_1	R_2

133

Another Example

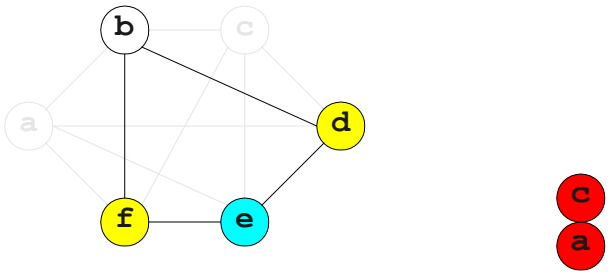


Registers

R_0	R_1	R_2

134

Another Example

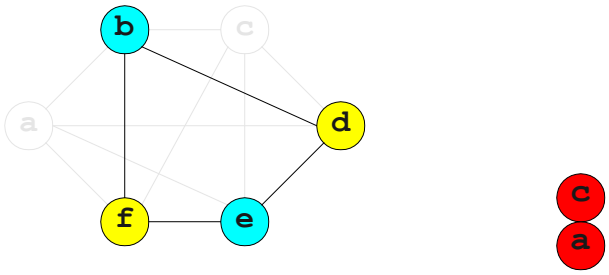


Registers

R_0	R_1	R_2

135

Another Example

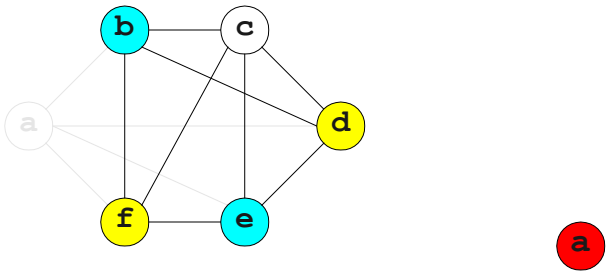


Registers

R_0	R_1	R_2

136

Another Example

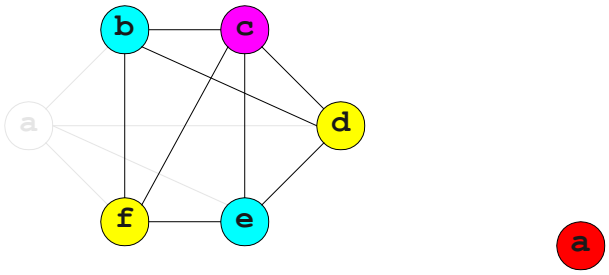


Registers

R_0	R_1	R_2

137

Another Example

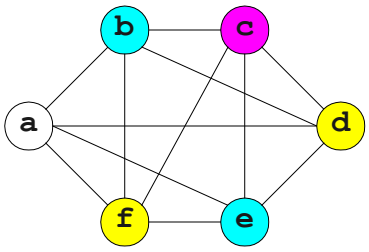


Registers

R_0	R_1	R_2

138

Another Example

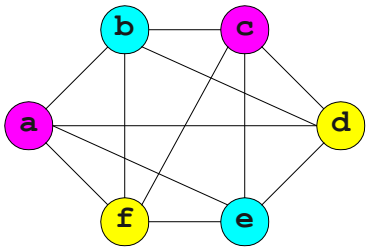


Registers

R_0	R_1	R_2

139

Another Example

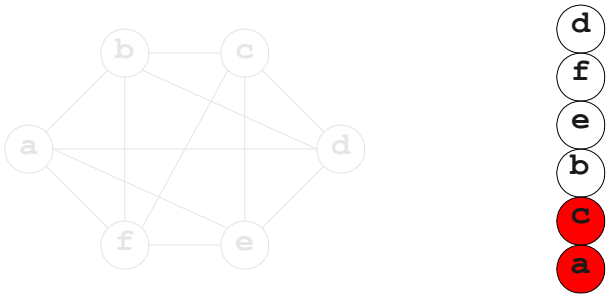


Registers

R_0	R_1	R_2

140

Another Example

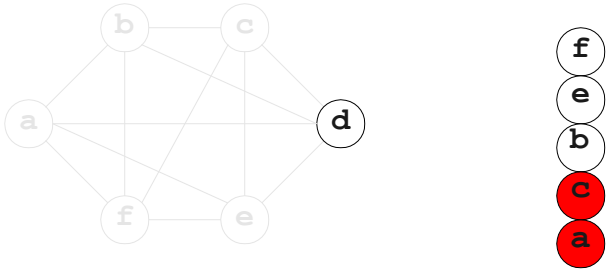


Registers

R_0	R_1	R_2

141

Another Example

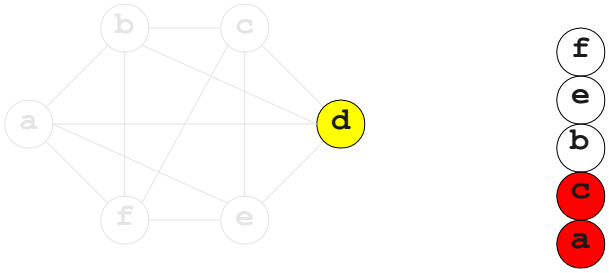


Registers

R_0	R_1	R_2

142

Another Example

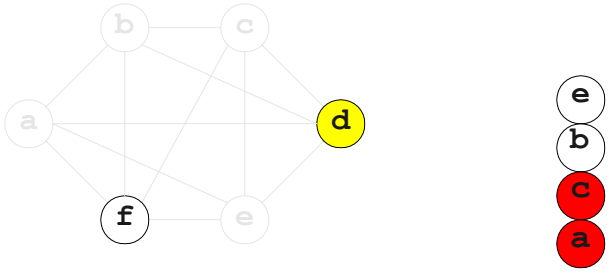


Registers

R_0	R_1	R_2

143

Another Example

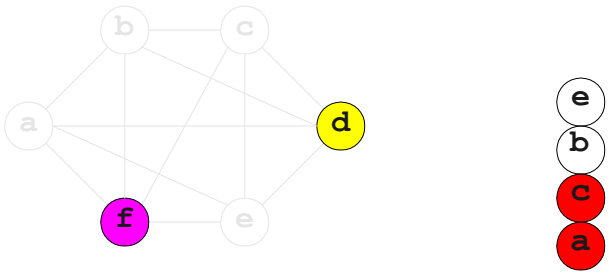


Registers

R_0	R_1	R_2

144

Another Example

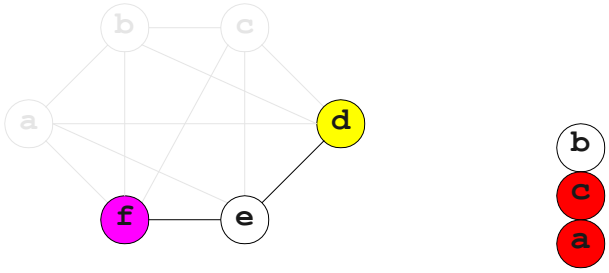


Registers

R_0	R_1	R_2

145

Another Example

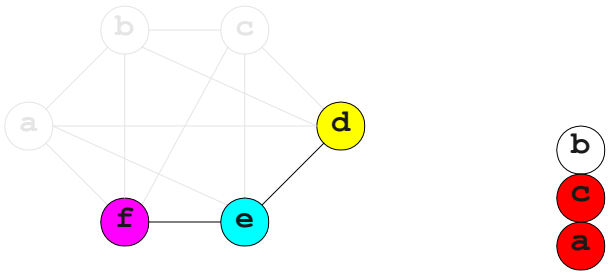


Registers

R_0	R_1	R_2

146

Another Example

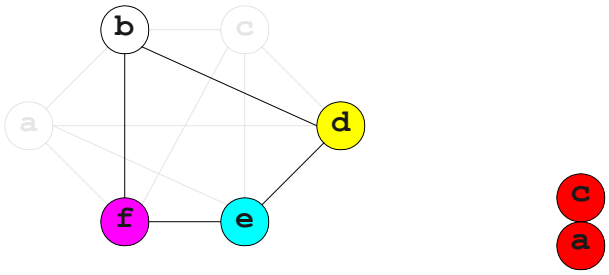


Registers

R_0	R_1	R_2

147

Another Example

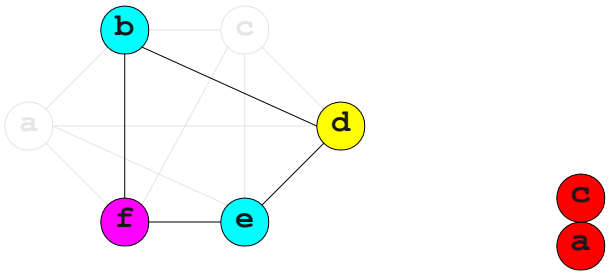


Registers

R_0	R_1	R_2

148

Another Example

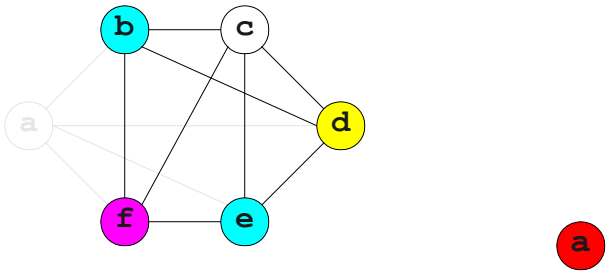


Registers

R_0	R_1	R_2

149

Another Example

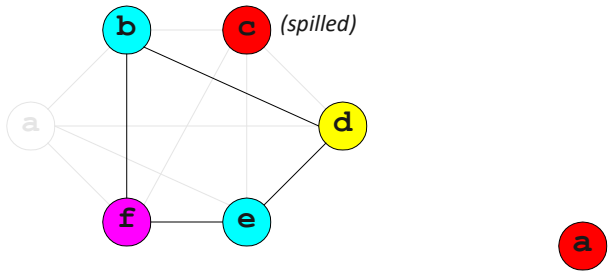


Registers

R_0	R_1	R_2

150

Another Example

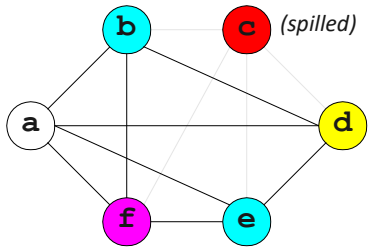


Registers

R_0	R_1	R_2

151

Another Example

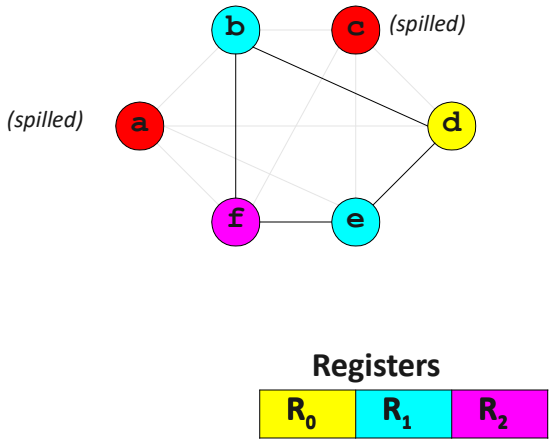


Registers

R_0	R_1	R_2

152

Another Example



153

Chaitin's Algorithm

- Advantages:
  - For many control-flow graphs, finds an excellent assignment of variables to registers.
  - When distinguishing variables by use, produces a precise RIG.
  - Often used in production compilers like GCC.
- Disadvantages:
  - Core approach based on the NP-hard graph coloring problem.
  - Heuristic may produce pathologically worst-case assignments.

154

Summary of Register Allocation

- Critical step in all optimizing compilers.
- The **linear scan** algorithm uses **live intervals** to greedily assign variables to registers.
  - Often used in JIT compilers due to efficiency.
- **Chaitin's algorithm** uses the **register interference graph** (based on **live ranges**) and **graph coloring** to assign registers.
  - The basis for the technique used in GCC.

155